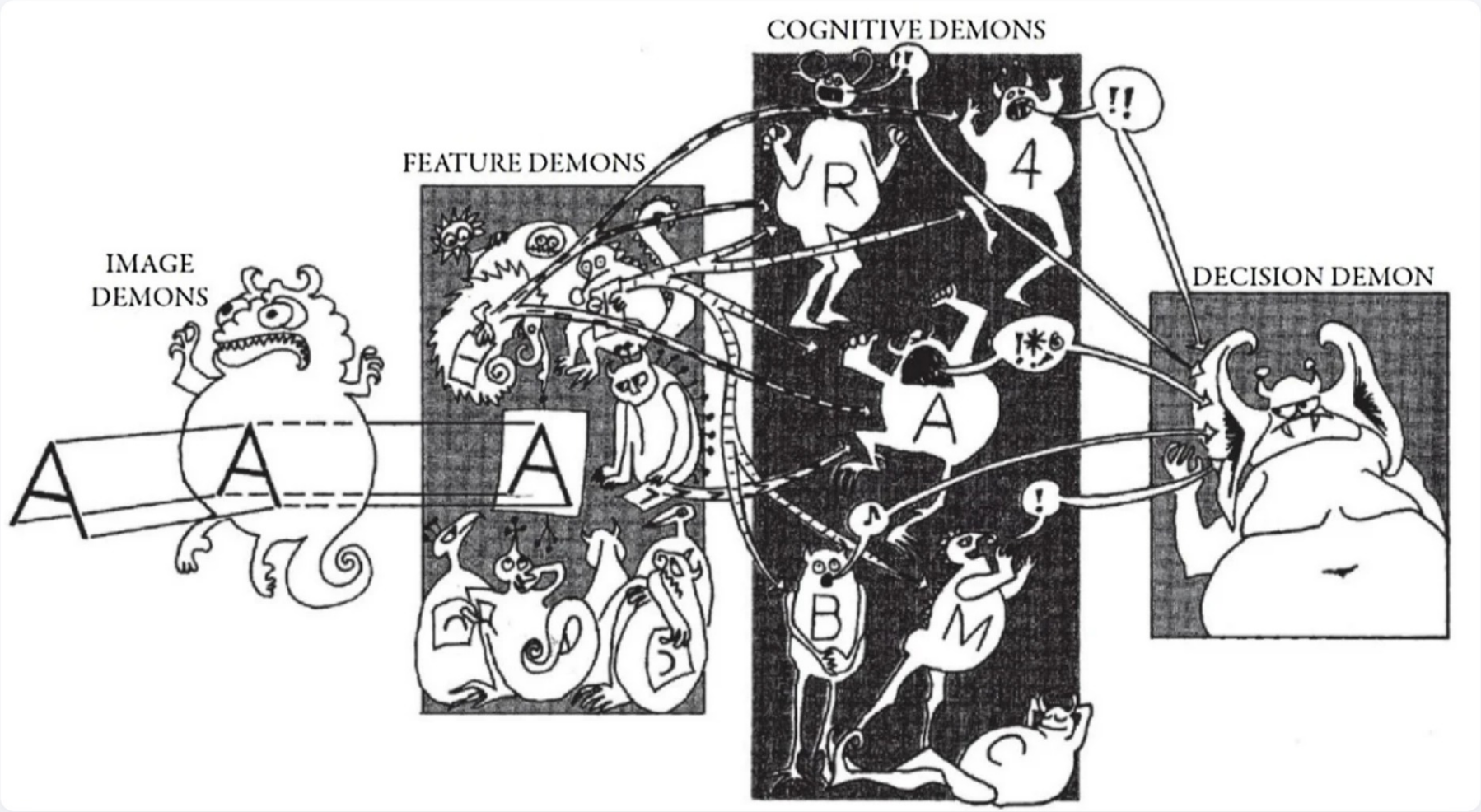


Selfridge's Pandemonium (1959): layers of "demons" that each shout when they detect their feature. Drawing from Lindsay & Norman, Human Information Processing.

MACHINE LEARNING

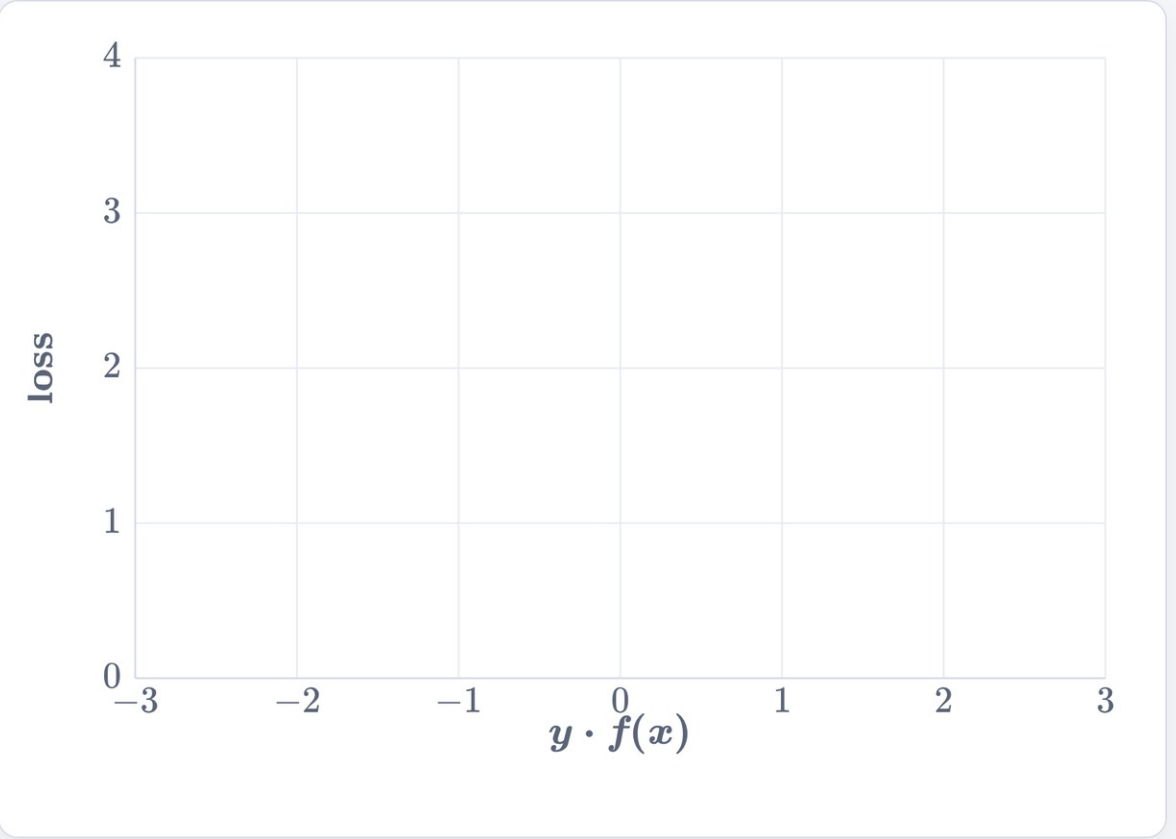
Neural networks

Part 1: from one neuron to a network.

LAST TIME

Losses for a classifier

- A score $f(\boldsymbol{x})$ and a label $y = \pm 1$. Predict $+1$ when $f(\boldsymbol{x}) > 0$.

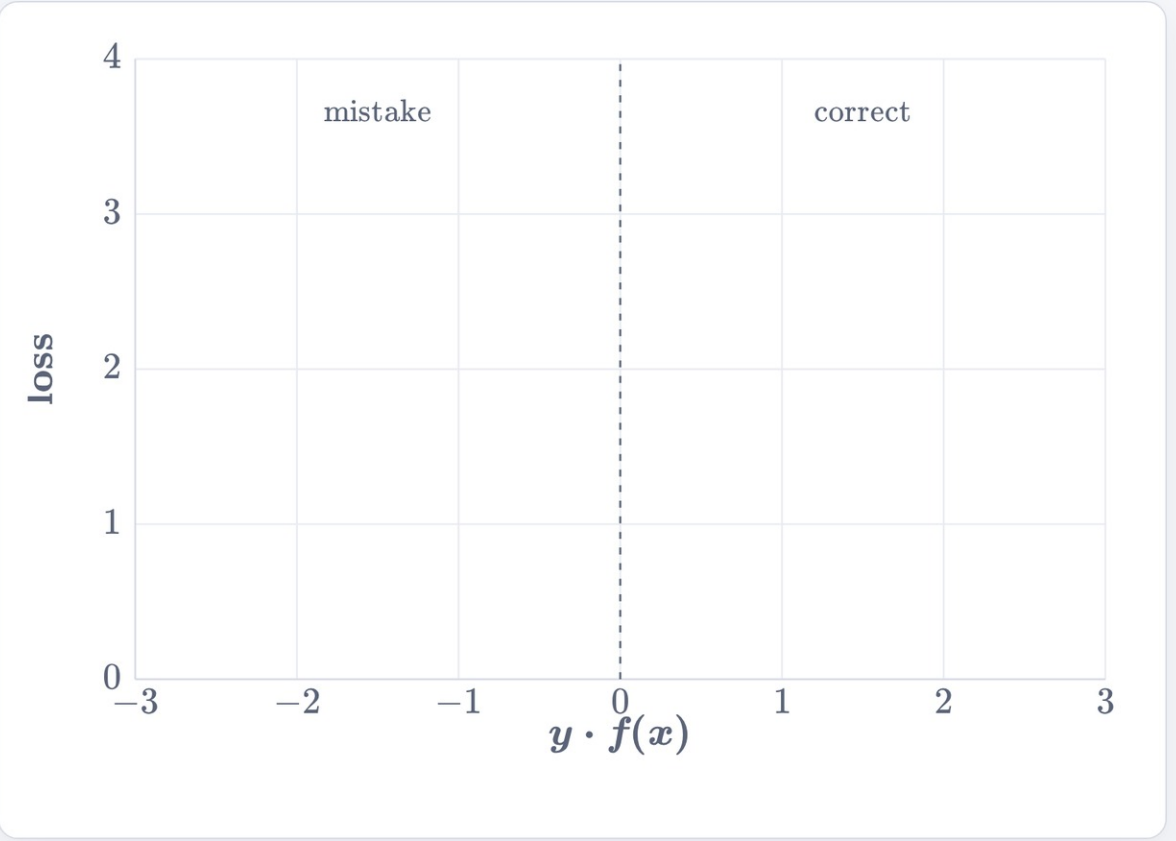


LAST TIME

Losses for a classifier

- A score $f(\boldsymbol{x})$ and a label $y = \pm 1$. Predict $+1$ when $f(\boldsymbol{x}) > 0$.
- Multiplying by the label turns both correct cases into $y\, f(\boldsymbol{x}) > 0$:

y	$f(\boldsymbol{x})$	$y\, f(\boldsymbol{x})$	
$+1$	> 0	> 0	**correct**
$+1$	< 0	< 0	**mistake**
-1	< 0	> 0	**correct**
-1	> 0	< 0	**mistake**



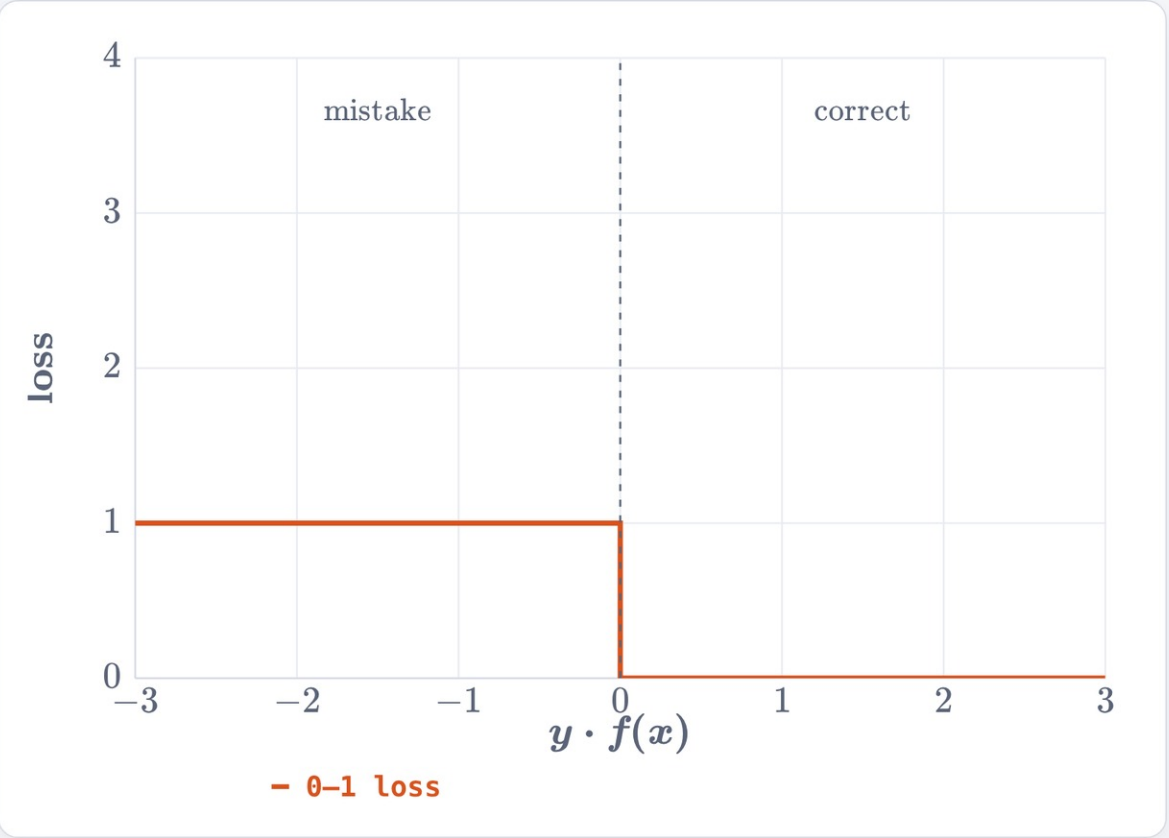
LAST TIME

Losses for a classifier

- A score $f(\boldsymbol{x})$ and a label $y = \pm 1$. Predict $+1$ when $f(\boldsymbol{x}) > 0$.
- Multiplying by the label turns both correct cases into $y\, f(\boldsymbol{x}) > 0$:

y	$f(\boldsymbol{x})$	$y\, f(\boldsymbol{x})$	
$+1$	> 0	> 0	correct
$+1$	< 0	< 0	mistake
-1	< 0	> 0	correct
-1	> 0	< 0	mistake

- **0–1 loss**: $\mathbf{1}\left[y\, f(\boldsymbol{x}) \leq 0\right]$, which is 1 for a mistake and 0 otherwise.



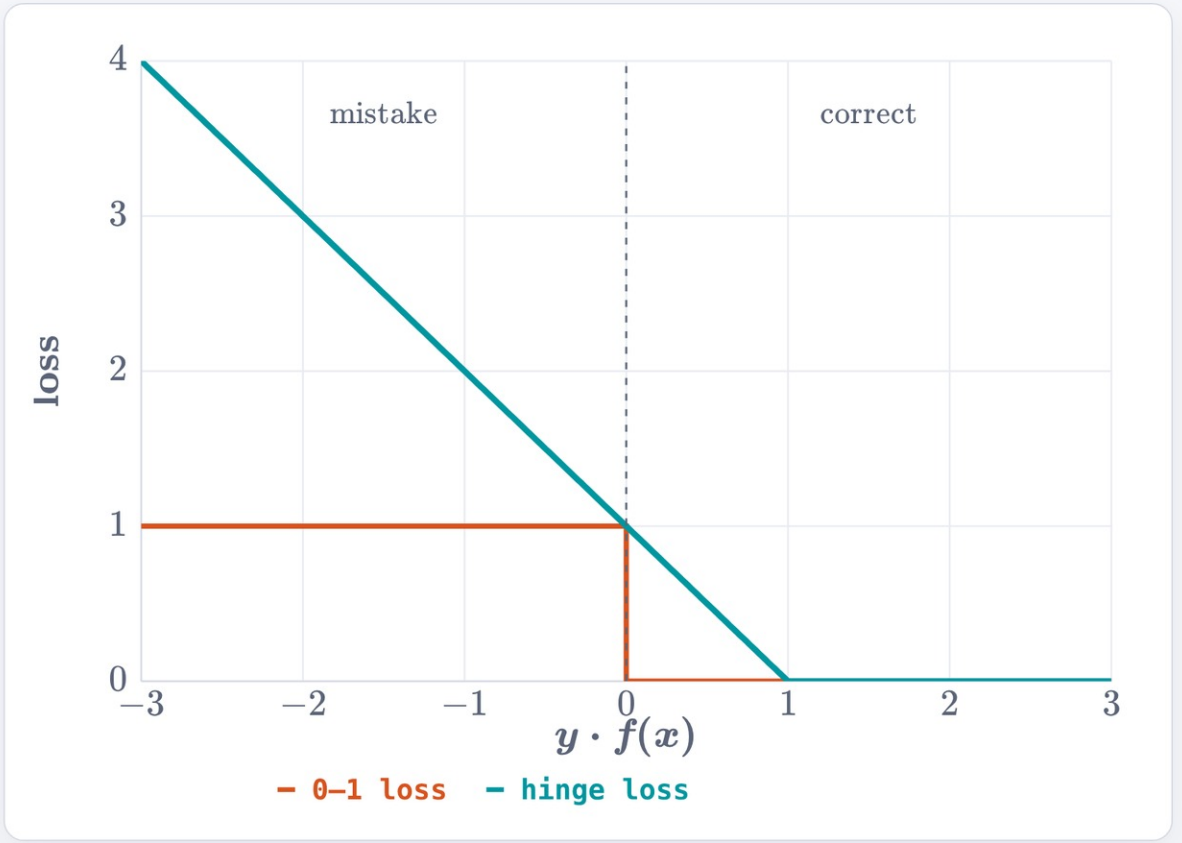
LAST TIME

Losses for a classifier

- A score $f(\boldsymbol{x})$ and a label $y = \pm 1$. Predict $+1$ when $f(\boldsymbol{x}) > 0$.
- Multiplying by the label turns both correct cases into $y\, f(\boldsymbol{x}) > 0$:

y	$f(\boldsymbol{x})$	$y\, f(\boldsymbol{x})$	
$+1$	> 0	> 0	**correct**
$+1$	< 0	< 0	**mistake**
-1	< 0	> 0	**correct**
-1	> 0	< 0	**mistake**

- **0–1 loss**: $\mathbf{1}\left[y\, f(\boldsymbol{x}) \leq 0\right]$, which is 1 for a mistake and 0 otherwise.
- **Hinge loss** (SVM): $\max\left(0, 1 - y\, f(\boldsymbol{x})\right)$.



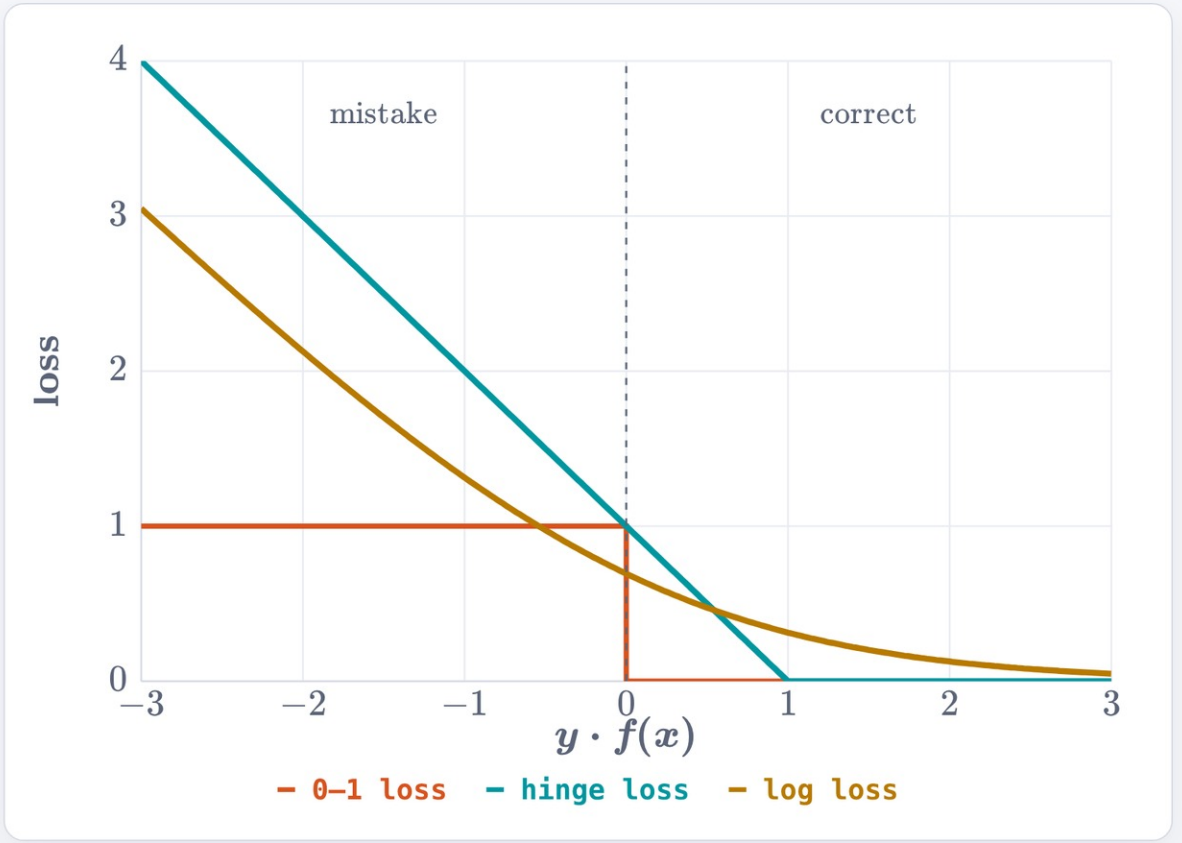
LAST TIME

Losses for a classifier

- A score $f(\boldsymbol{x})$ and a label $y = \pm 1$. Predict $+1$ when $f(\boldsymbol{x}) > 0$.
- Multiplying by the label turns both correct cases into $y\, f(\boldsymbol{x}) > 0$:

y	$f(\boldsymbol{x})$	$y\, f(\boldsymbol{x})$	
$+1$	> 0	> 0	correct
$+1$	< 0	< 0	mistake
-1	< 0	> 0	correct
-1	> 0	< 0	mistake

- **0–1 loss**: $\mathbf{1}\left[y\, f(\boldsymbol{x}) \leq 0\right]$, which is 1 for a mistake and 0 otherwise.
- **Hinge loss** (SVM): $\max\left(0, 1 - y\, f(\boldsymbol{x})\right)$.
- **Log loss** (logistic regression): $\log\left(1 + e^{-y f(\boldsymbol{x})}\right)$.



LAST TIME

Logistic regression

Model

$f(\boldsymbol{x}) := \boldsymbol{w} \cdot \boldsymbol{x} + b$

$p(y = +1 \mid \boldsymbol{x}) = \sigma\big(f(\boldsymbol{x})\big)$

One score, squashed into a probability.

LAST TIME

Logistic regression

Model

$f(\boldsymbol{x}) := \boldsymbol{w} \cdot \boldsymbol{x} + b$

$p(y = +1 \mid \boldsymbol{x}) = \sigma\big(f(\boldsymbol{x})\big)$

One score, squashed into a probability.

Loss

$\mathcal{L}(\boldsymbol{\theta}; D) = \sum_i \log\big(1 + e^{-y_i f(\boldsymbol{x}_i)}\big)$

Log loss on the training data $D = \{(\boldsymbol{x}_i, y_i)\}$, with $y_i = \pm 1$.

LAST TIME

Logistic regression

Model

$f(\boldsymbol{x}) := \boldsymbol{w} \cdot \boldsymbol{x} + b$

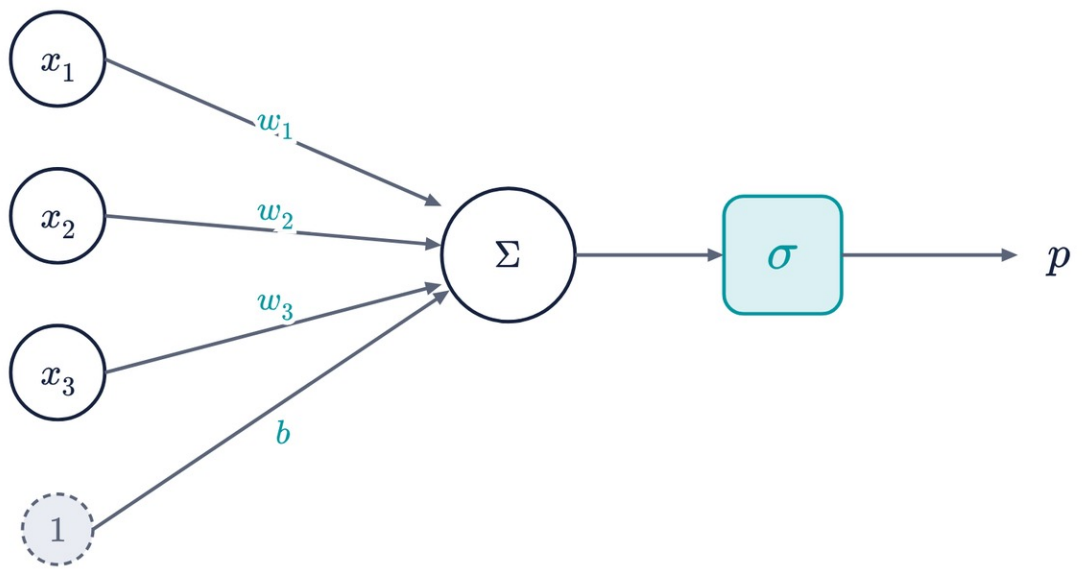
$p(y = +1 \mid \boldsymbol{x}) = \sigma\big(f(\boldsymbol{x})\big)$

One score, squashed into a probability.

Loss

$\mathcal{L}(\boldsymbol{\theta}; D) = \sum_i \log\big(1 + e^{-y_i f(\boldsymbol{x}_i)}\big)$

Log loss on the training data $D = \{(\boldsymbol{x}_i, y_i)\}$, with $y_i = \pm 1$.

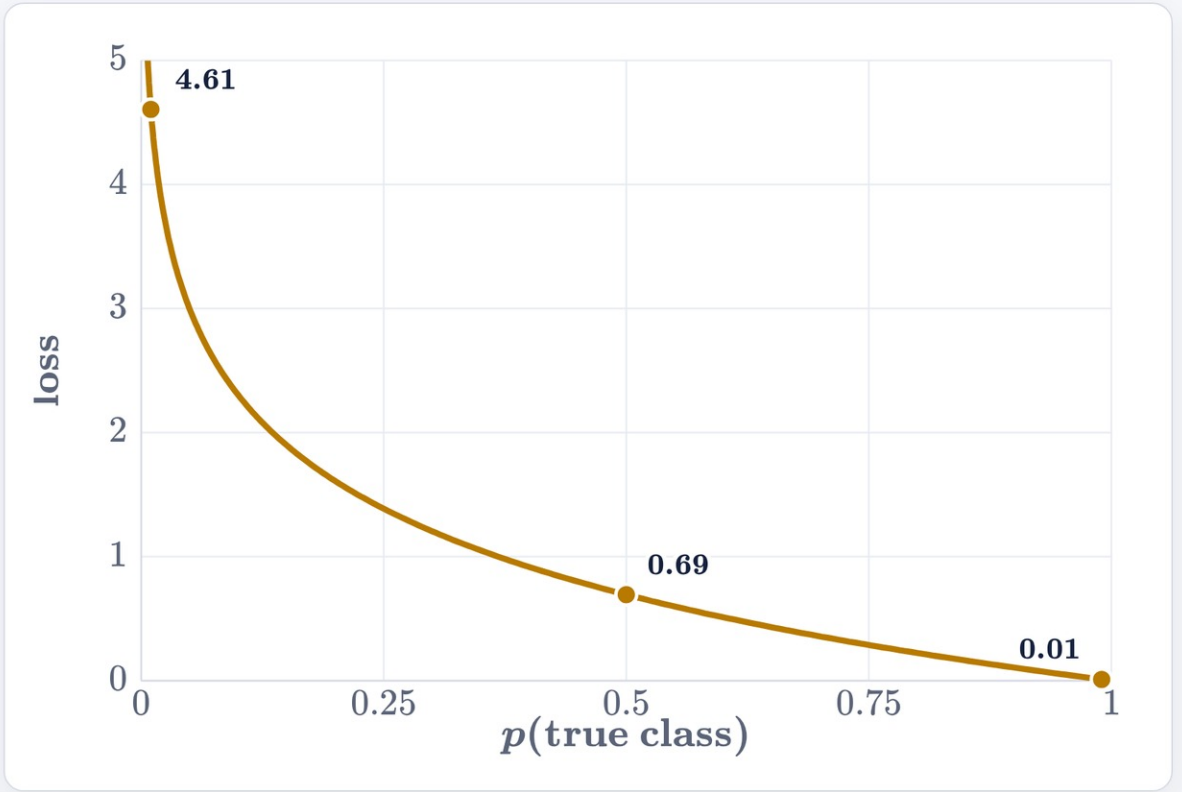


LAST TIME

Confident and wrong costs a lot

- The log loss of one point is

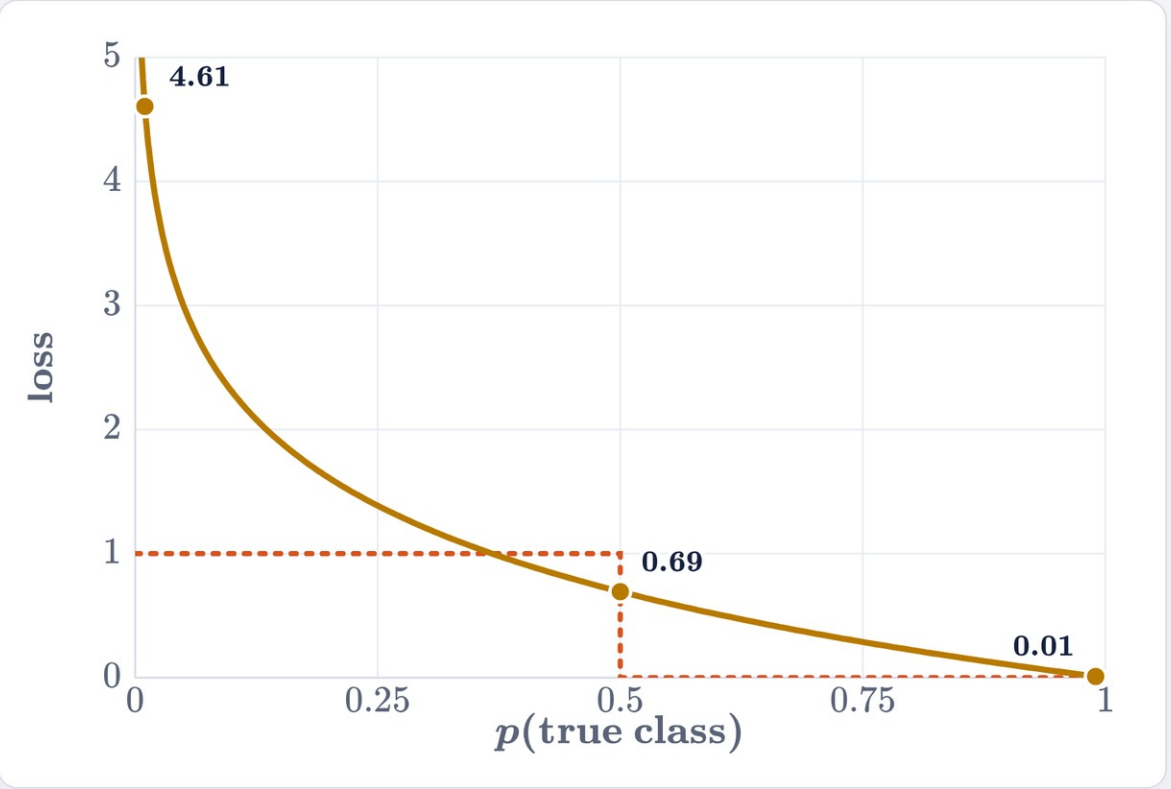
  $-\log p(\text{true class})$



LAST TIME

Confident and wrong costs a lot

- The log loss of one point is
  $-\log p(\text{true class})$
- The 0–1 loss (dotted) only asks whether $p(\text{true class})$ is above 0.5.



FROM NEURON TO MODEL

A very simplified neuron

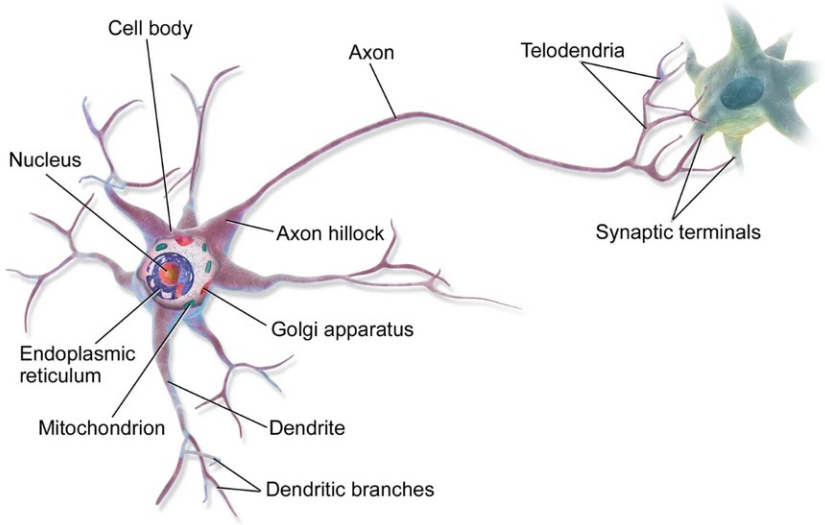


Illustration: BruceBlaus, "Blausen 0657 MultipolarNeuron", Wikimedia Commons, CC BY 3.0.

FROM NEURON TO MODEL

A very simplified neuron

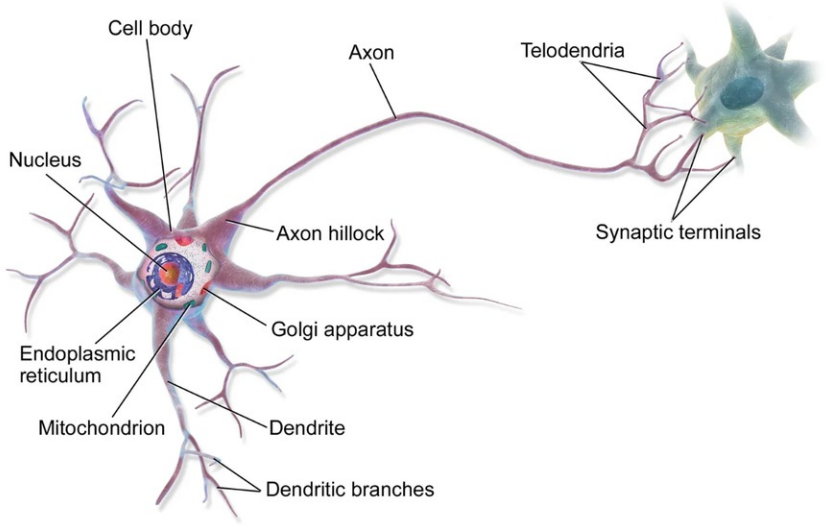
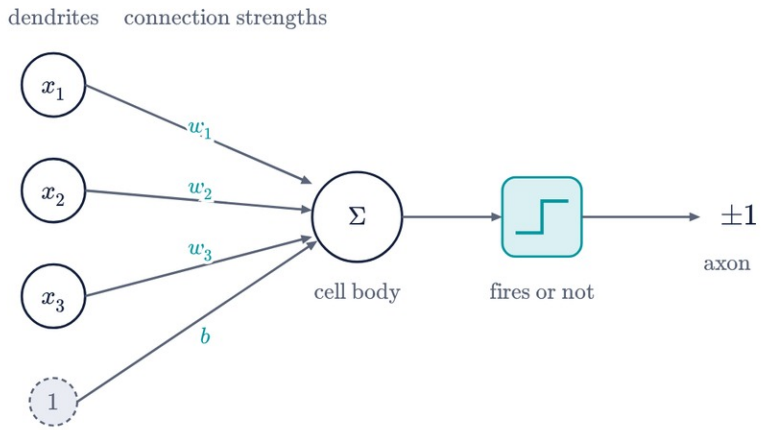


Illustration: BruceBlaus, "Blausen 0657 MultipolarNeuron", Wikimedia Commons, CC BY 3.0.



FROM NEURON TO MODEL

A very simplified neuron

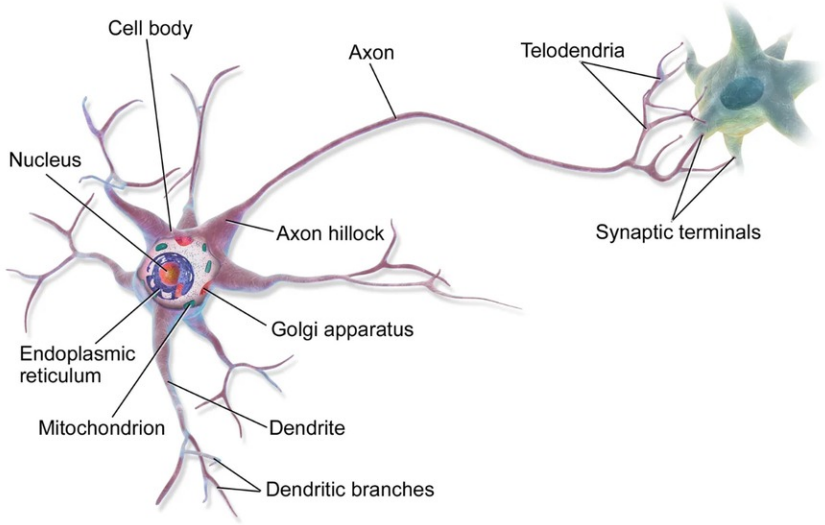
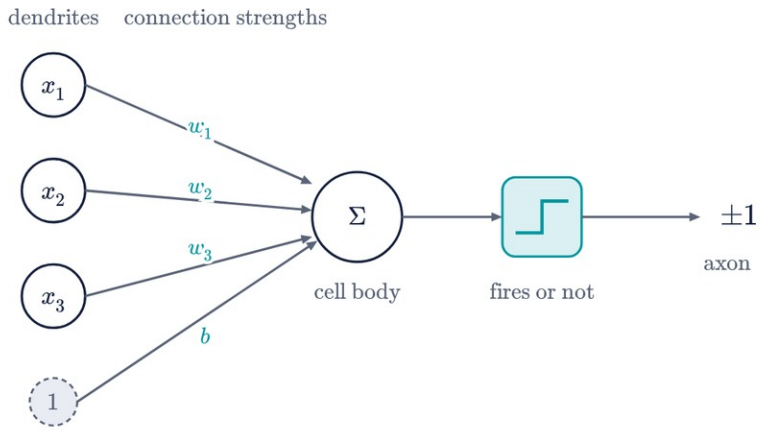


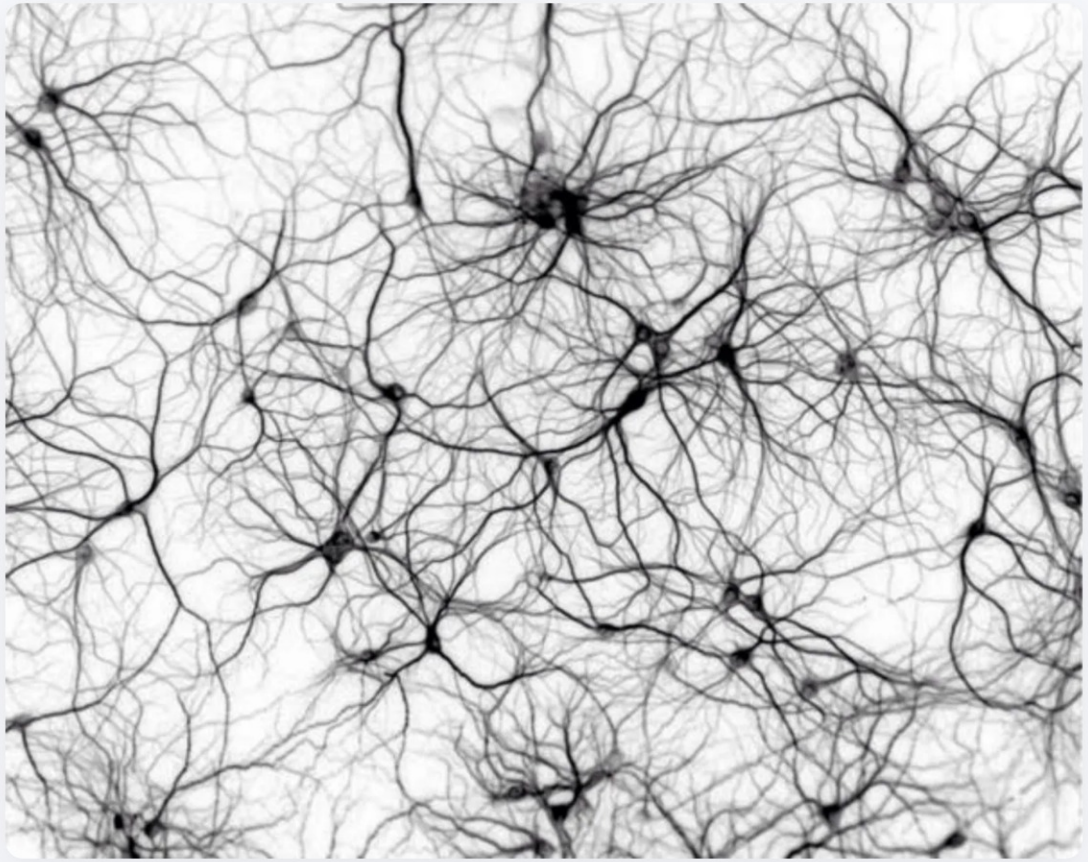
Illustration: BruceBlaus, "Blausen 0657 MultipolarNeuron", Wikimedia Commons, CC BY 3.0.



W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity", Bulletin of Mathematical Biophysics 5 (1943), 115–133.
F. Rosenblatt, "The perceptron: a probabilistic model for information storage and organization in the brain", Psychological Review 65(6) (1958), 386–408.

WHERE THE NAME COMES FROM

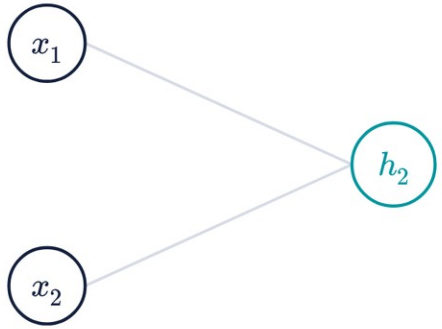
Networks of neurons



A HIDDEN LAYER

Combining multiple neurons

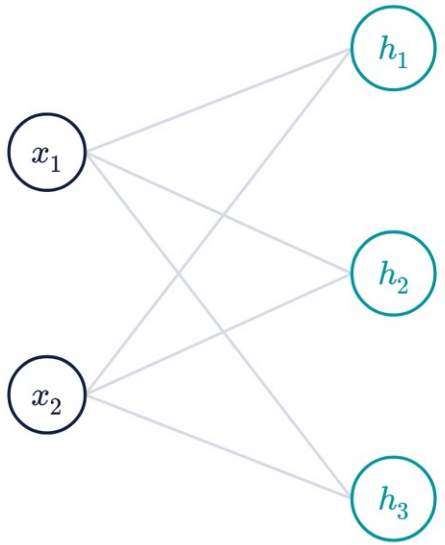
- Start from one neuron.



A HIDDEN LAYER

Combining multiple neurons

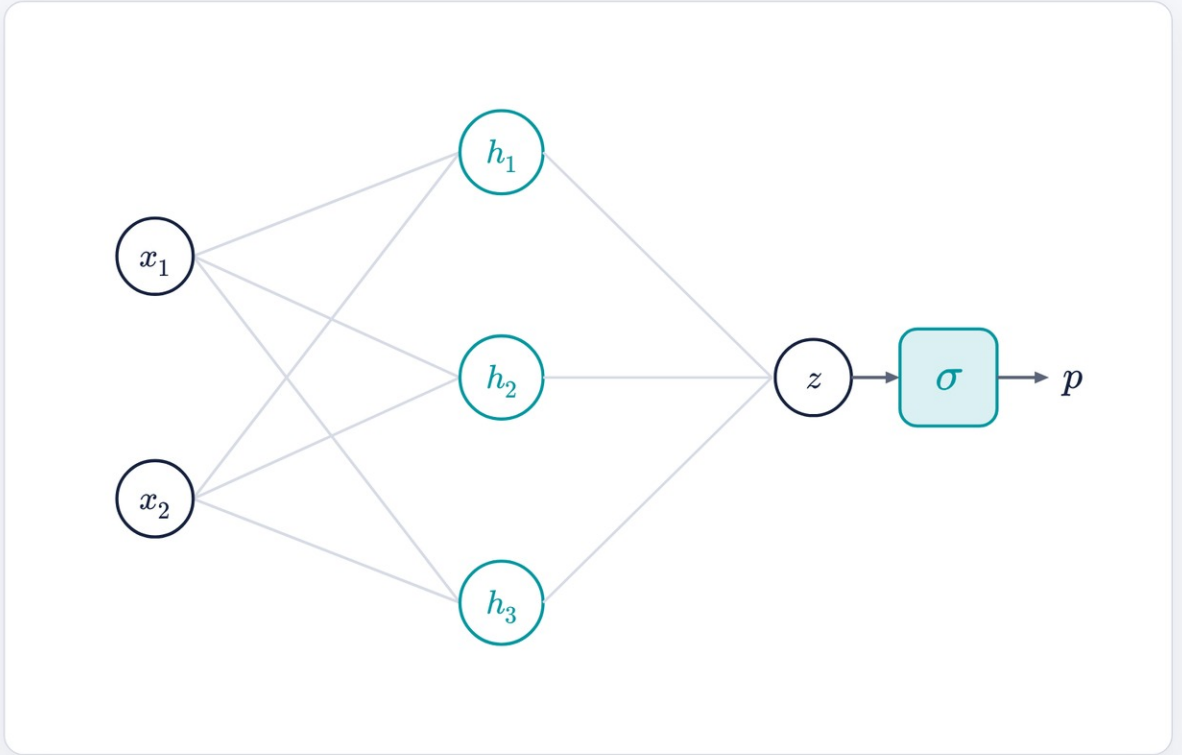
- Start from one neuron.
- Copy it: each hidden neuron is a logistic regression on the inputs.



A HIDDEN LAYER

Combining multiple neurons

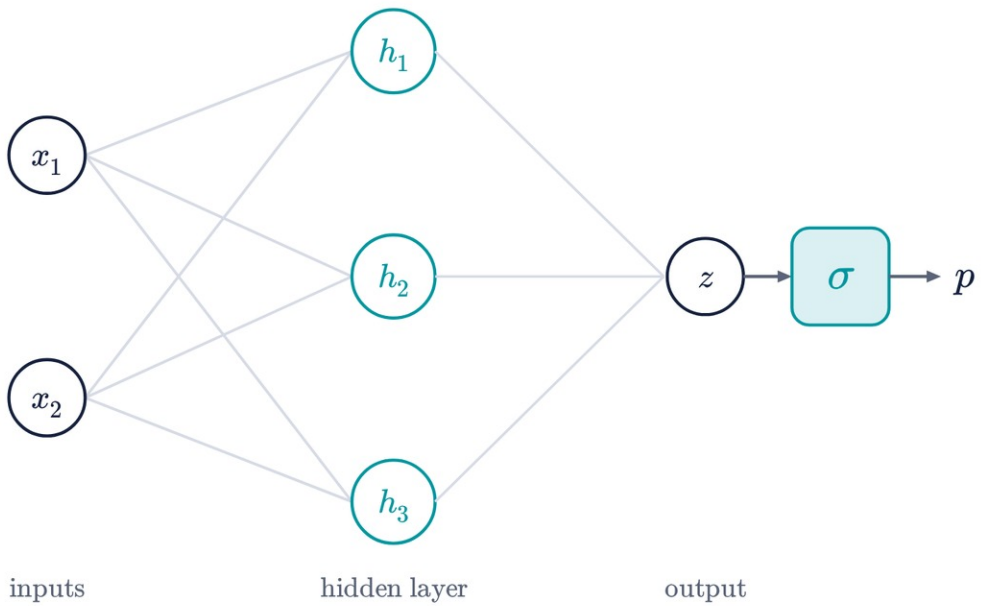
- Start from one neuron.
- Copy it: each hidden neuron is a logistic regression on the inputs.
- Feed the hidden neurons to one output neuron: a logistic regression on the hidden neurons, trained with the log loss.



A HIDDEN LAYER

Combining multiple neurons

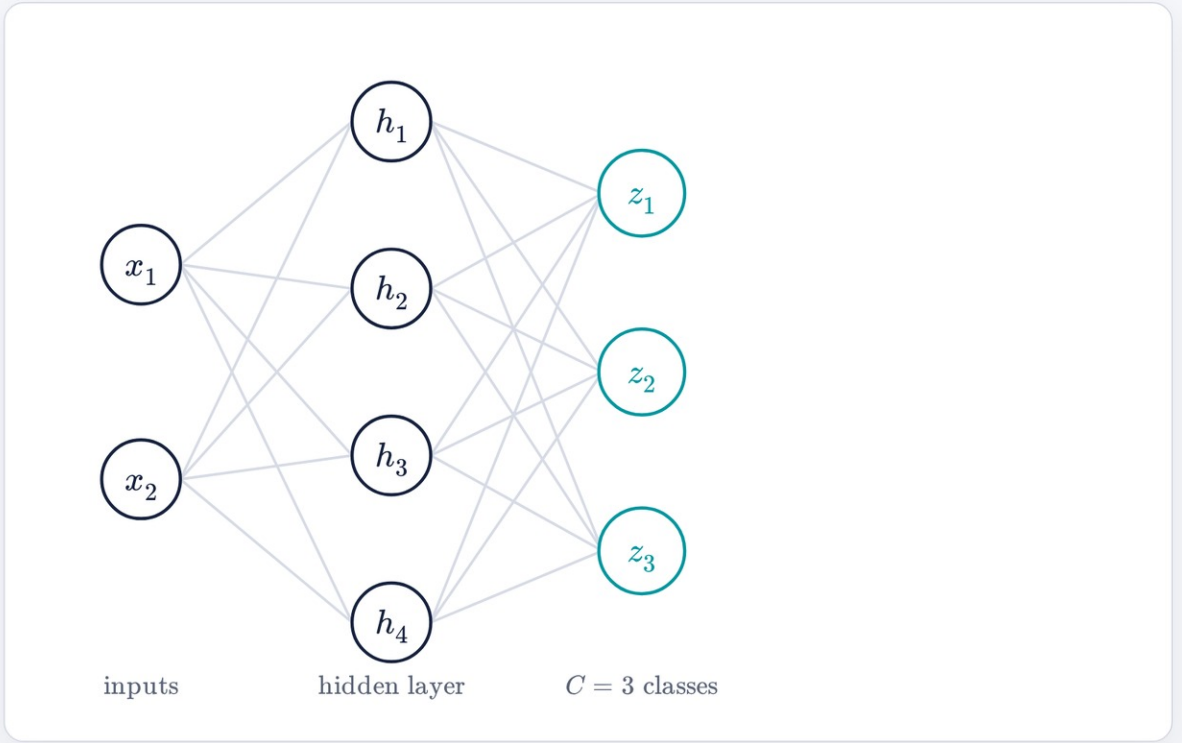
- Start from one neuron.
- Copy it: each hidden neuron is a logistic regression on the inputs.
- Feed the hidden neurons to one output neuron: a logistic regression on the hidden neurons, trained with the log loss.
- Feed-forward and fully connected: a **multilayer perceptron** (MLP).



MORE THAN TWO CLASSES

Softmax

- One output score per class: z_1, z_2, z_3 for $C = 3$ classes.



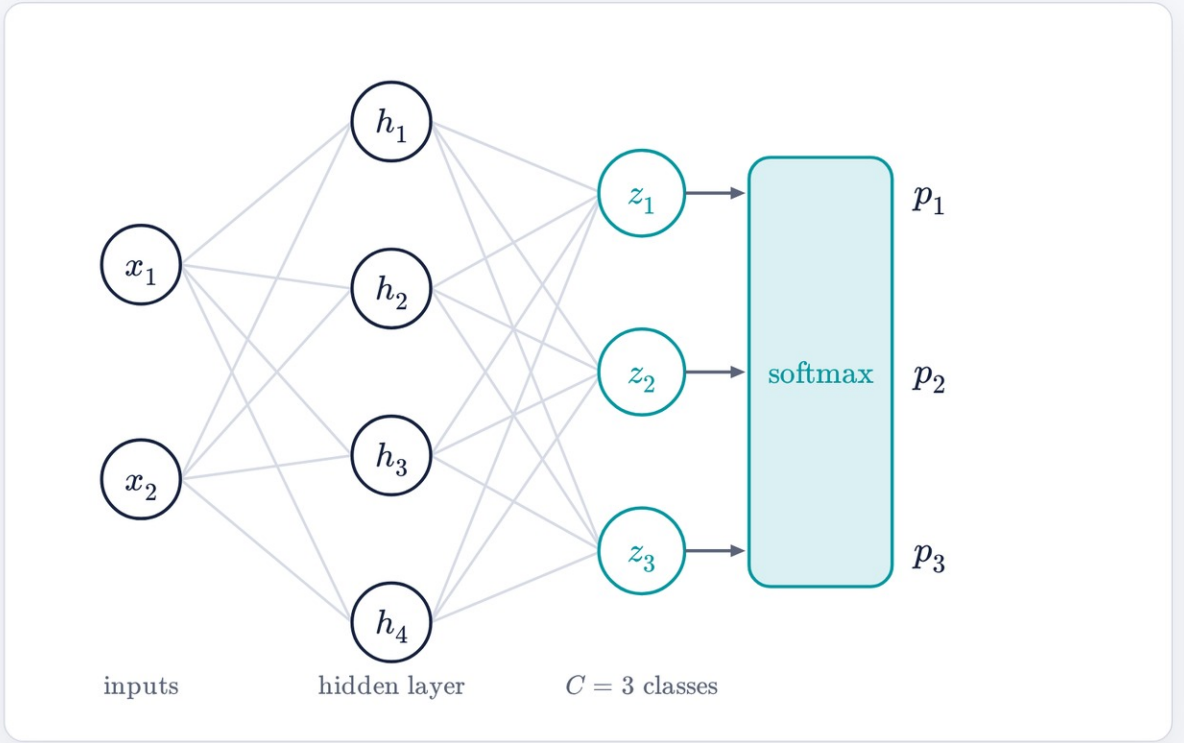
MORE THAN TWO CLASSES

Softmax

- One output score per class: z_1, z_2, z_3 for $C = 3$ classes.
- The **softmax** turns the scores into probabilities:

  $$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

  They are positive and sum to 1.



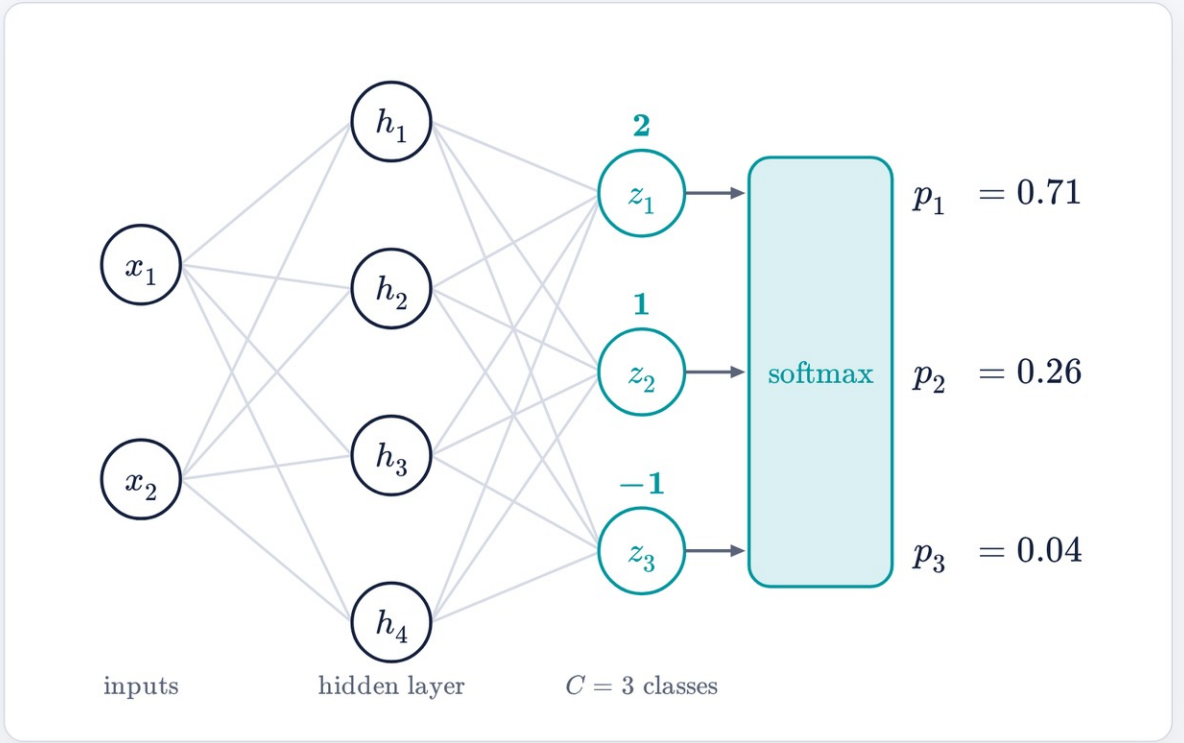
MORE THAN TWO CLASSES

Softmax

- One output score per class: z_1, z_2, z_3 for $C = 3$ classes.
- The **softmax** turns the scores into probabilities:

  $$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

  They are positive and sum to 1.
- Example: $\boldsymbol{z} = (2, 1, -1)$ gives $p_1 = 0.71$, $p_2 = 0.26$, $p_3 = 0.04$.



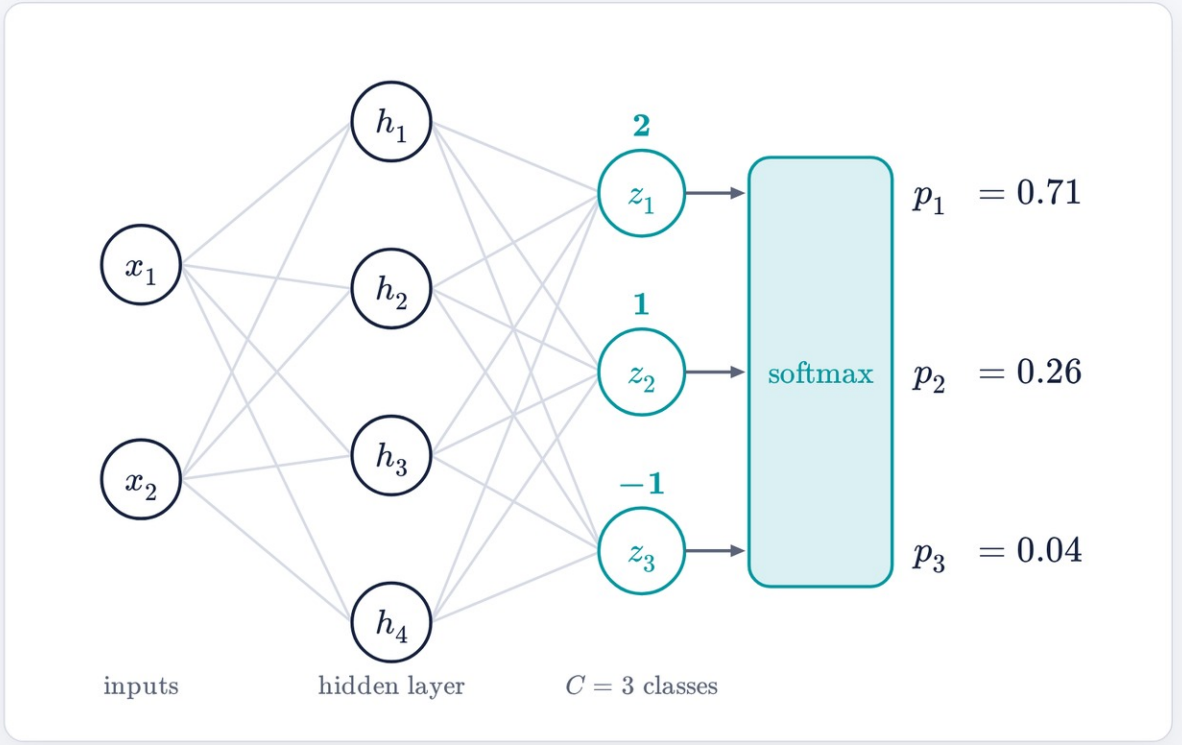
MORE THAN TWO CLASSES

Softmax

- One output score per class: z_1, z_2, z_3 for $C = 3$ classes.
- The **softmax** turns the scores into probabilities:

  $$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

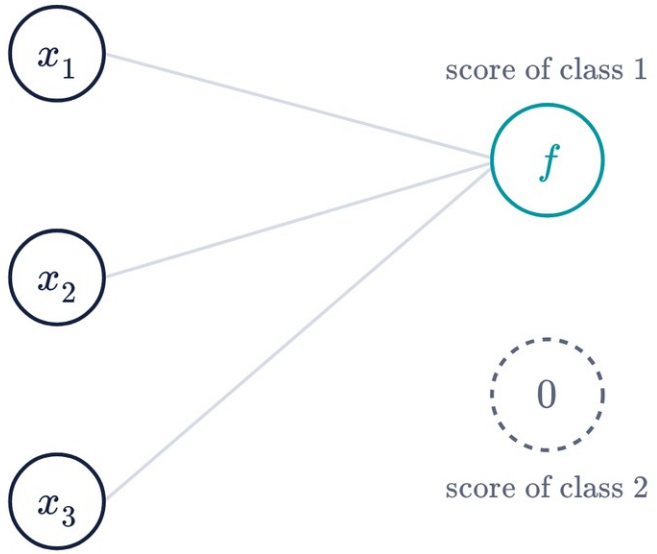
  They are positive and sum to 1.
- Example: $\boldsymbol{z} = (2, 1, -1)$ gives $p_1 = 0.71$, $p_2 = 0.26$, $p_3 = 0.04$.
- Predict the class with the largest p_k.



TWO CLASSES

Softmax with two classes is the sigmoid

- With two classes, give them the scores $(f, 0)$.

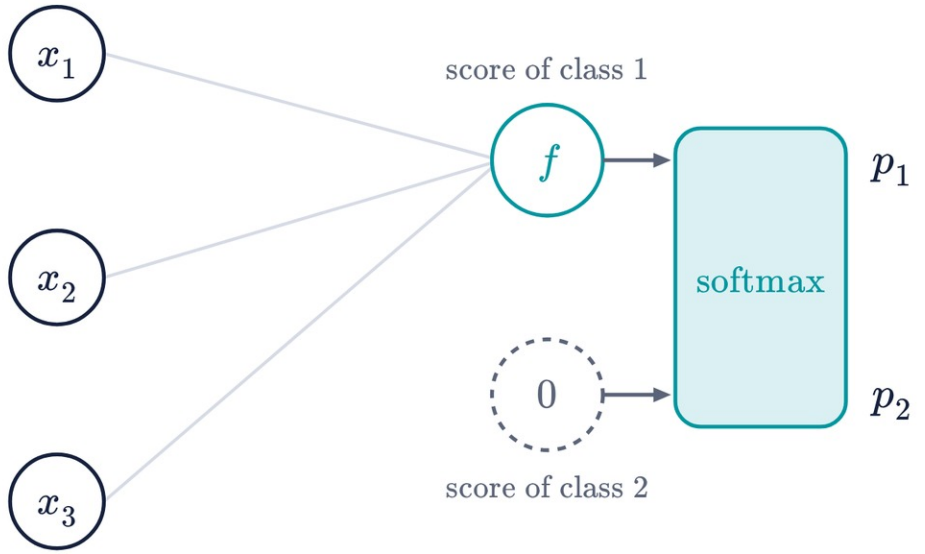


TWO CLASSES

Softmax with two classes is the sigmoid

- With two classes, give them the scores $(f, 0)$.
- The softmax of these scores:

$$p_1 = \frac{e^f}{e^f + e^0} = \frac{1}{1 + e^{-f}} = \sigma(f)$$



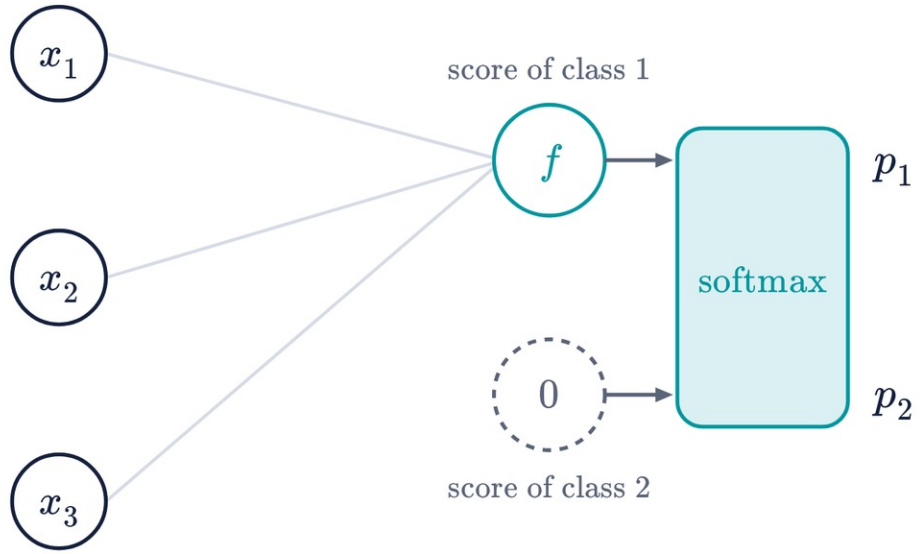
TWO CLASSES

Softmax with two classes is the sigmoid

- With two classes, give them the scores $(f, 0)$.
- The softmax of these scores:

$$p_1 = \frac{e^f}{e^f + e^0} = \frac{1}{1 + e^{-f}} = \sigma(f)$$

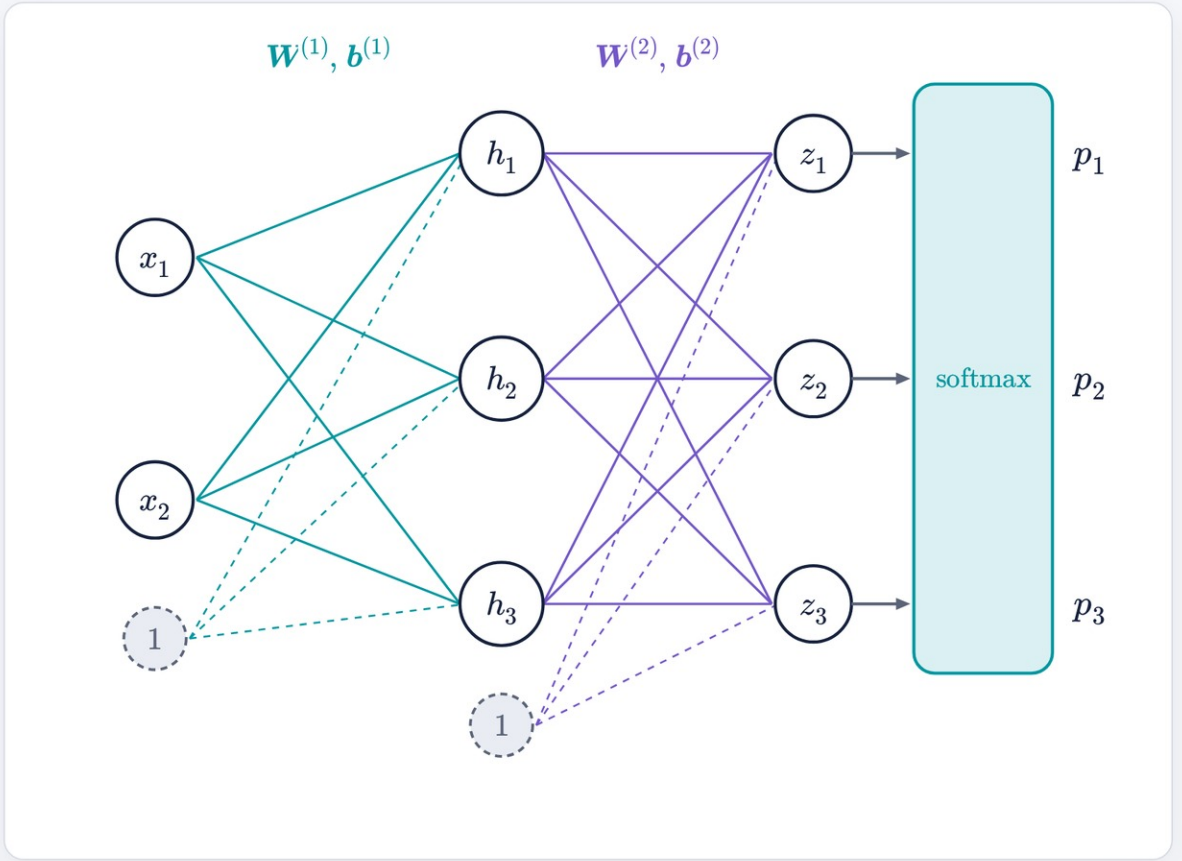
- So one output neuron with a sigmoid is a two-class softmax: last lecture's logistic regression.



WHAT THE NETWORK COMPUTES

Forward propagation

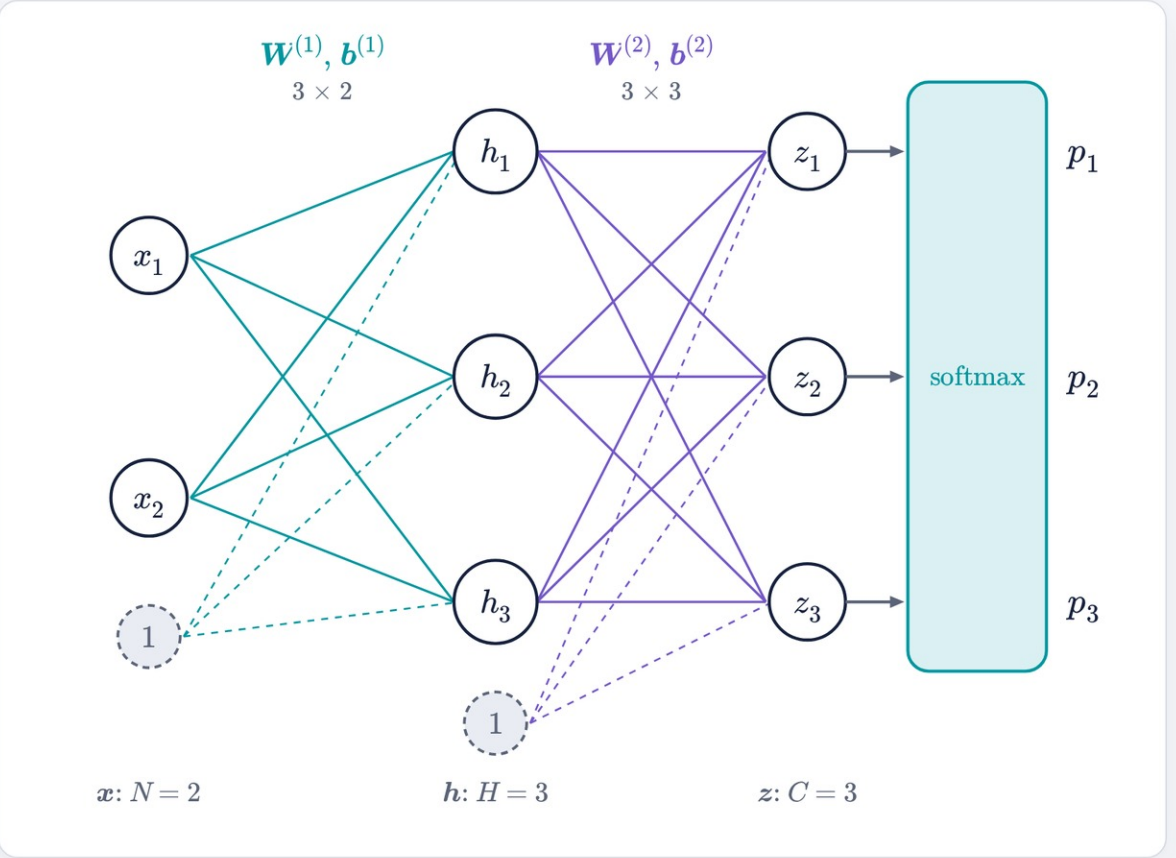
$$\boldsymbol{h} = \sigma(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$$
$$\boldsymbol{z} = \boldsymbol{W}^{(2)}\boldsymbol{h} + \boldsymbol{b}^{(2)}$$
$$\boldsymbol{p} = \text{softmax}(\boldsymbol{z})$$



Forward propagation

$$\boldsymbol{h} = \sigma(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$$
$$\boldsymbol{z} = \boldsymbol{W}^{(2)}\boldsymbol{h} + \boldsymbol{b}^{(2)}$$
$$\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$$

- Shapes: $\boldsymbol{x}$ has N entries, $\boldsymbol{W}^{(1)}$ is $H \times N$, $\boldsymbol{W}^{(2)}$ is $C \times H$. The parameters $\boldsymbol{\theta}$ are $\boldsymbol{W}^{(1)}$, $\boldsymbol{b}^{(1)}$, $\boldsymbol{W}^{(2)}$ and $\boldsymbol{b}^{(2)}$. The superscript is the layer.



Forward propagation

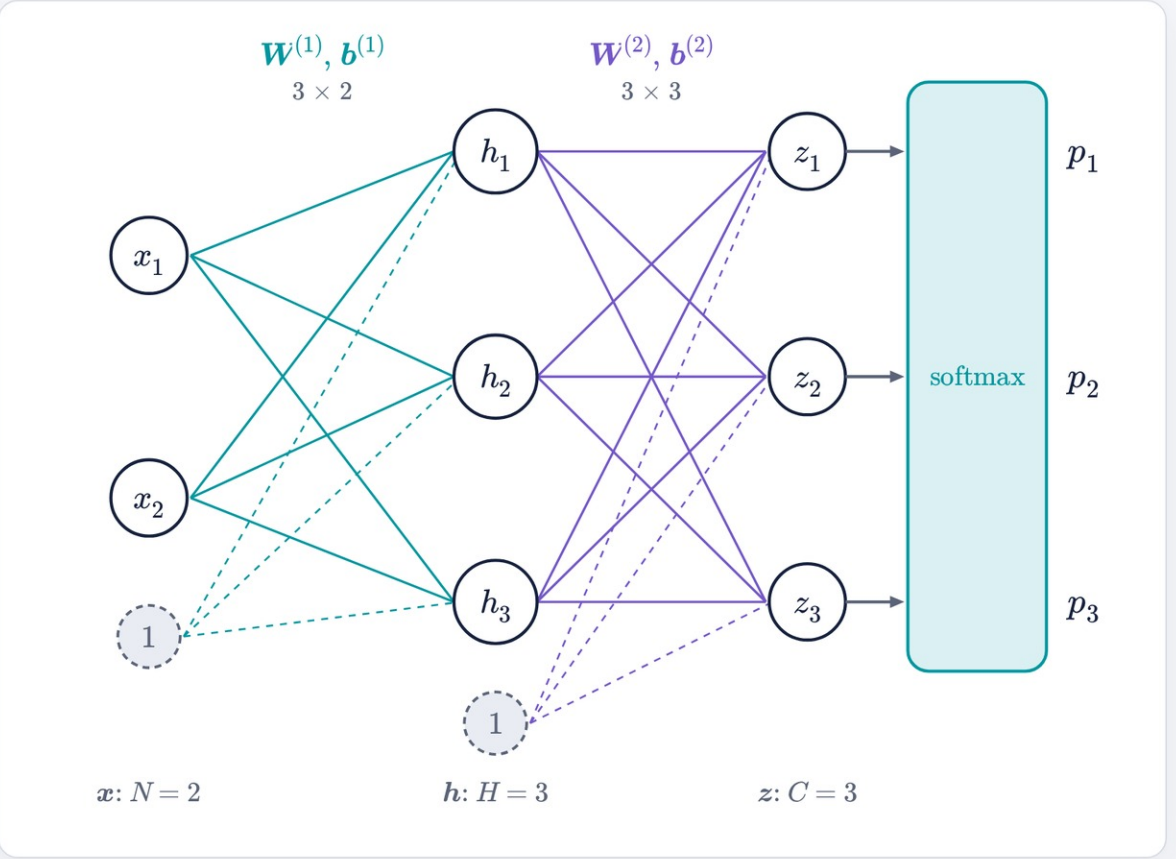
$$\boldsymbol{h} = \sigma(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$$
$$\boldsymbol{z} = \boldsymbol{W}^{(2)}\boldsymbol{h} + \boldsymbol{b}^{(2)}$$
$$\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$$

- Shapes: $\boldsymbol{x}$ has N entries, $\boldsymbol{W}^{(1)}$ is $H \times N$, $\boldsymbol{W}^{(2)}$ is $C \times H$. The parameters $\boldsymbol{\theta}$ are $\boldsymbol{W}^{(1)}$, $\boldsymbol{b}^{(1)}$, $\boldsymbol{W}^{(2)}$ and $\boldsymbol{b}^{(2)}$. The superscript is the layer.
- Why a nonlinearity? Without one, the layers collapse into one linear layer:

  $\boldsymbol{W}^{(2)}(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}) + \boldsymbol{b}^{(2)}$

  $= (\boldsymbol{W}^{(2)}\boldsymbol{W}^{(1)})\,\boldsymbol{x} + (\boldsymbol{W}^{(2)}\boldsymbol{b}^{(1)} + \boldsymbol{b}^{(2)})$

  Any nonlinear function φ works; the sigmoid is one choice.



NOT ONLY SIGMOIDS

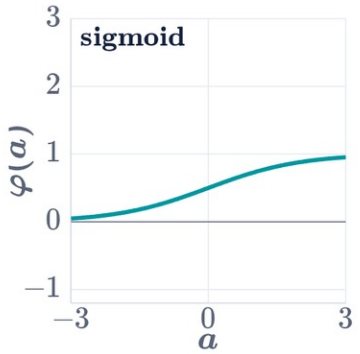
Activation functions

- The function each hidden neuron applies is its **activation function** φ:
  $\boldsymbol{h} = \varphi(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$

NOT ONLY SIGMOIDS

Activation functions

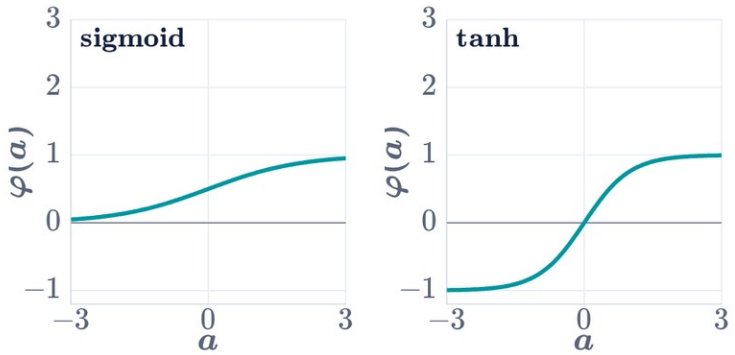
- The function each hidden neuron applies is its **activation function** φ:
  $\boldsymbol{h} = \varphi(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$
- Sigmoid, $\sigma(a) = 1/(1 + e^{-a})$: between 0 and 1.



NOT ONLY SIGMOIDS

Activation functions

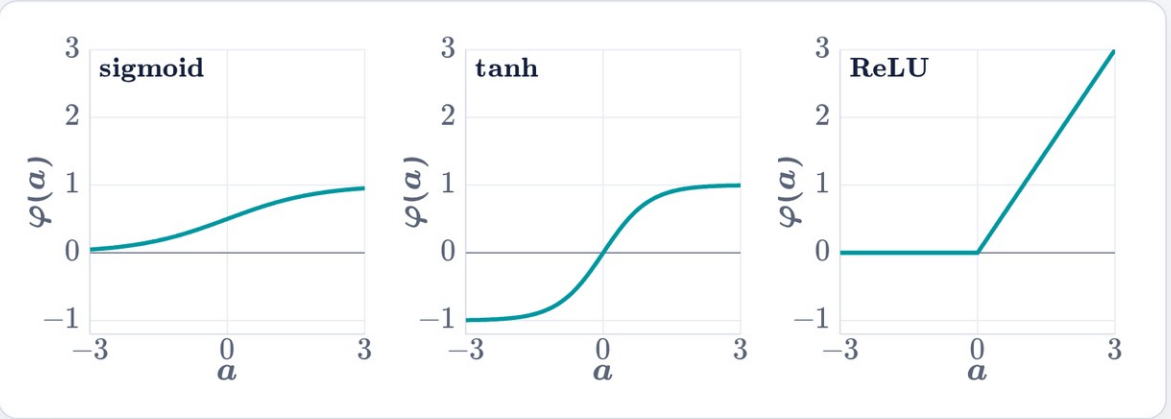
- The function each hidden neuron applies is its **activation function** φ:
  $\boldsymbol{h} = \varphi(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$
- Sigmoid, $\sigma(a) = 1/(1 + e^{-a})$: between 0 and 1.
- $\tanh(a)$: between -1 and 1, a rescaled sigmoid.



NOT ONLY SIGMOIDS

Activation functions

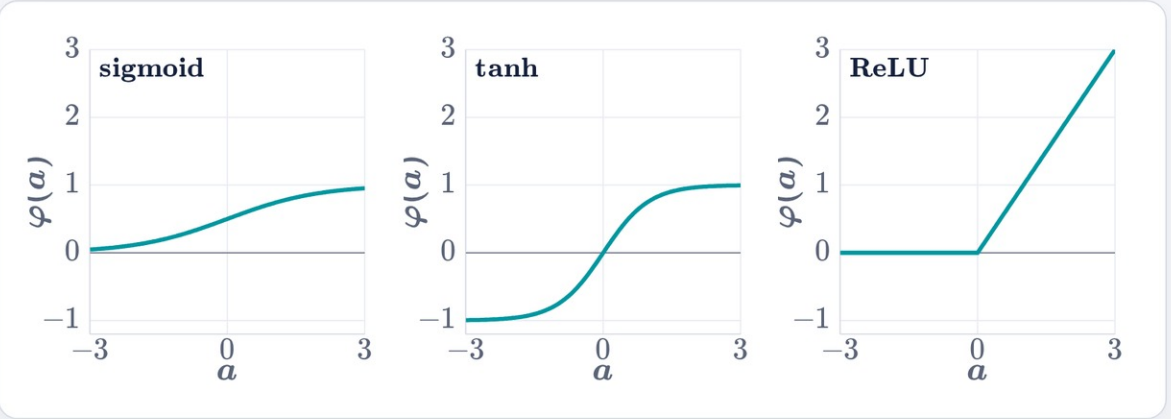
- The function each hidden neuron applies is its **activation function** φ:
  $\boldsymbol{h} = \varphi(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$
- Sigmoid, $\sigma(a) = 1/(1 + e^{-a})$: between 0 and 1.
- $\tanh(a)$: between -1 and 1, a rescaled sigmoid.
- ReLU, $\max(0, a)$: 0 for negative inputs, the input itself for positive ones. Widely used in deep networks.



NOT ONLY SIGMOIDS

Activation functions

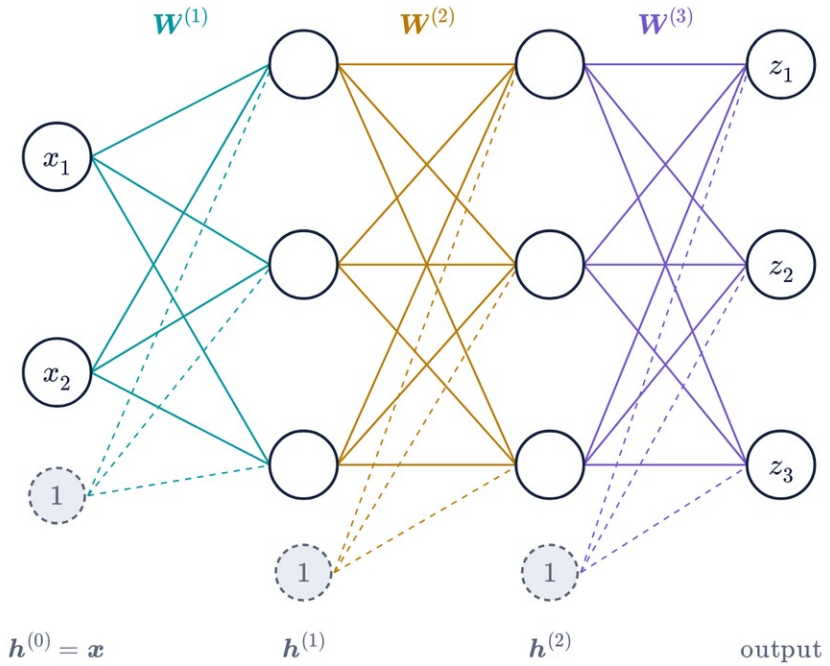
- The function each hidden neuron applies is its **activation function** φ:
  $\boldsymbol{h} = \varphi(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$
- Sigmoid, $\sigma(a) = 1/(1 + e^{-a})$: between 0 and 1.
- $\tanh(a)$: between -1 and 1, a rescaled sigmoid.
- ReLU, $\max(0, a)$: 0 for negative inputs, the input itself for positive ones. Widely used in deep networks.
- The output layer stays the same: the softmax, or σ for two classes.



MORE LAYERS

Deeper networks

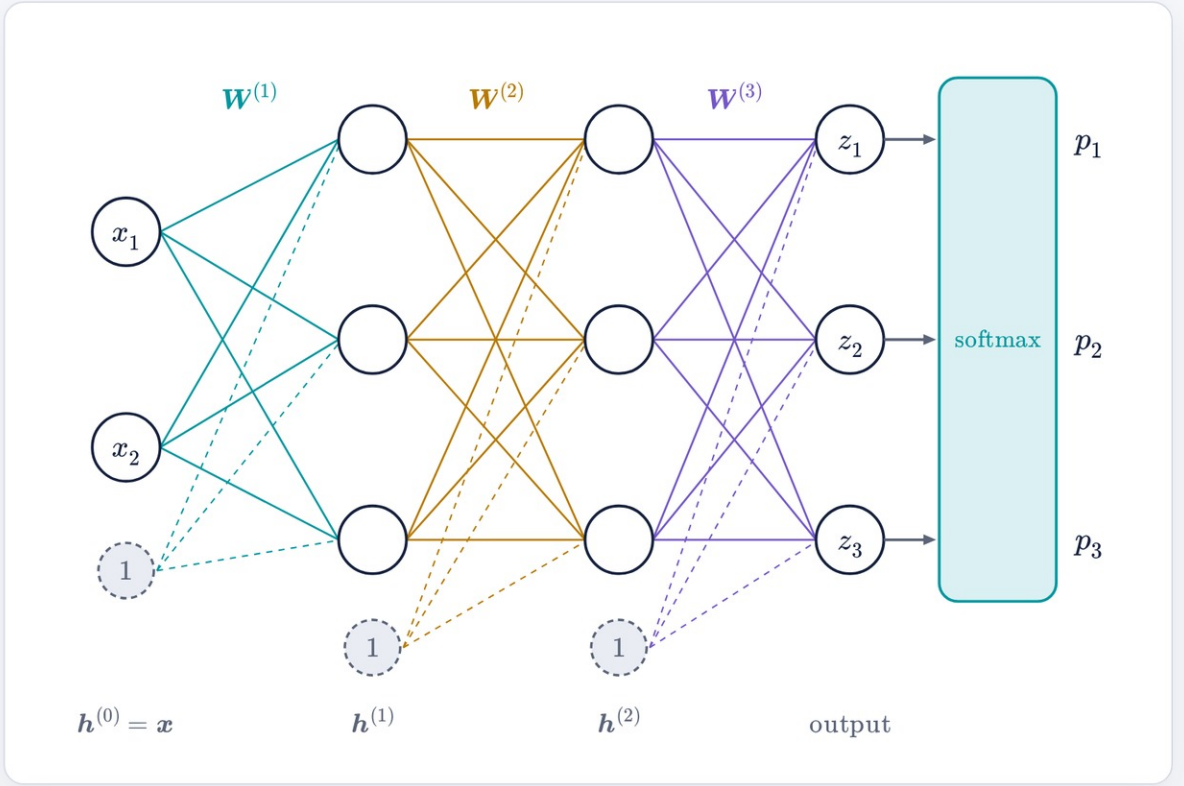
- Stack hidden layers. Each layer works on the output of the one before:
  $\boldsymbol{h}^{(l)} = \varphi\big(\boldsymbol{W}^{(l)}\boldsymbol{h}^{(l-1)} + \boldsymbol{b}^{(l)}\big)$
  for $l = 1, \ldots, L-1$, starting from $\boldsymbol{h}^{(0)} = \boldsymbol{x}$.



MORE LAYERS

Deeper networks

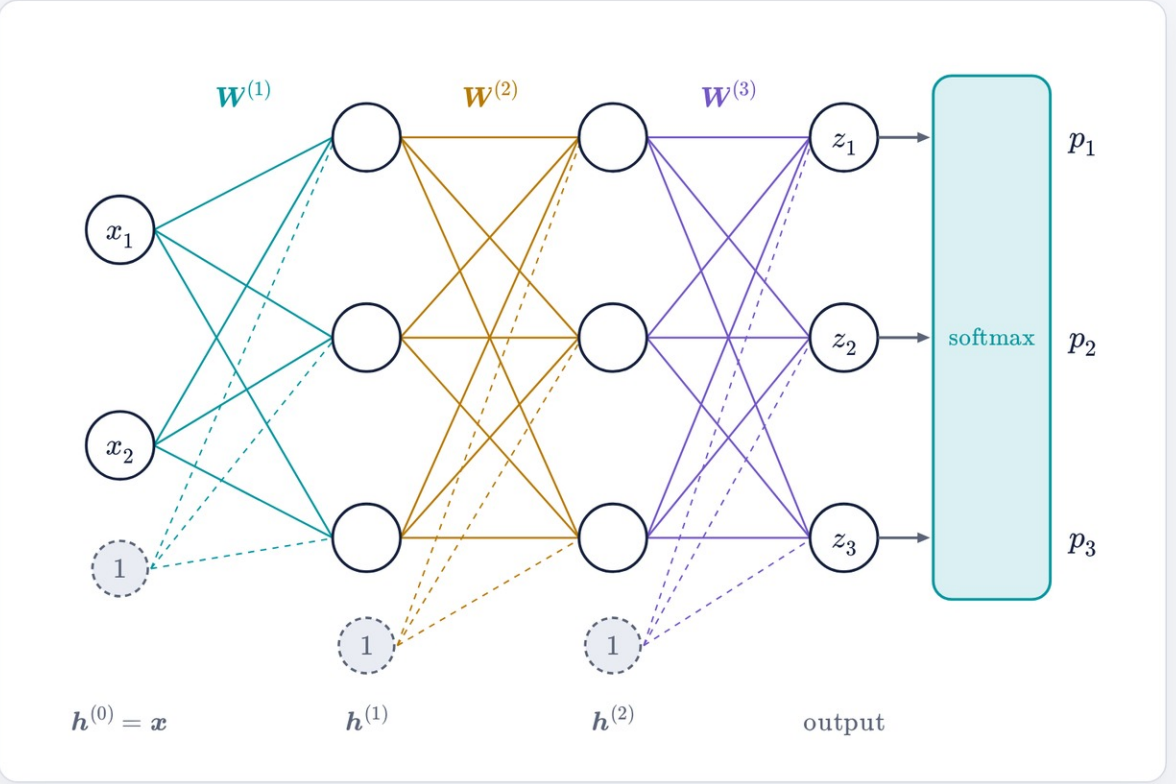
- Stack hidden layers. Each layer works on the output of the one before:
  $\boldsymbol{h}^{(l)} = \varphi\big(\boldsymbol{W}^{(l)}\boldsymbol{h}^{(l-1)} + \boldsymbol{b}^{(l)}\big)$
  for $l = 1, \dots, L-1$, starting from $\boldsymbol{h}^{(0)} = \boldsymbol{x}$.
- The last layer is the softmax:
  $\boldsymbol{p} = \mathrm{softmax}\big(\boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)}\big)$



MORE LAYERS

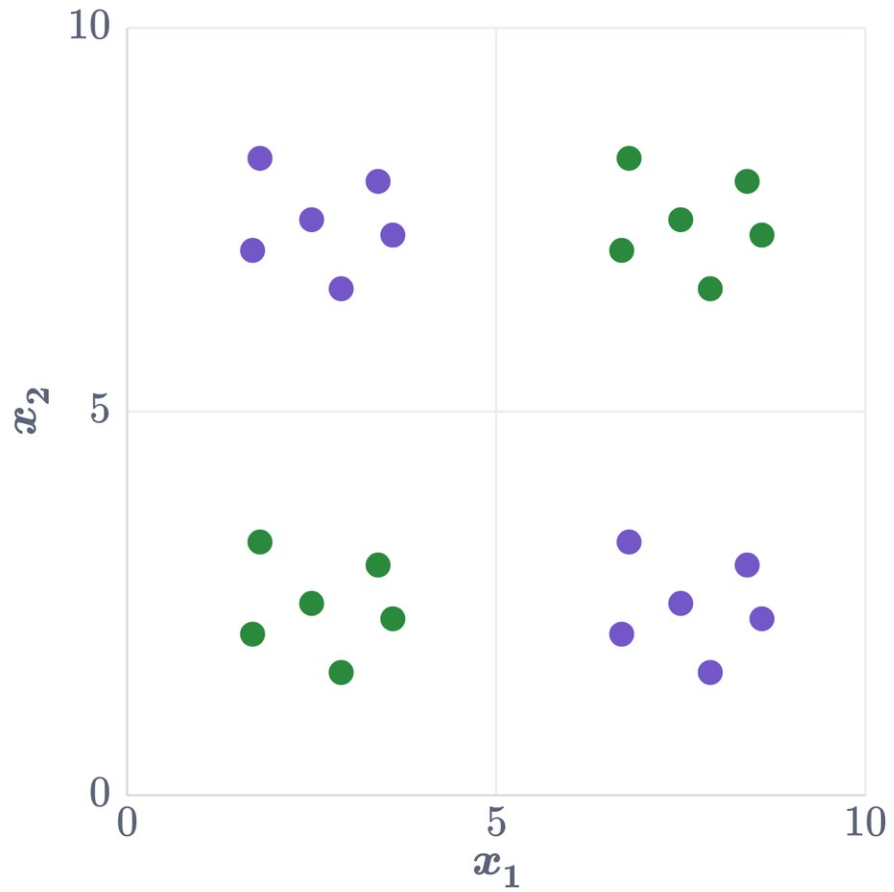
Deeper networks

- Stack hidden layers. Each layer works on the output of the one before:
  $\boldsymbol{h}^{(l)} = \varphi\big(\boldsymbol{W}^{(l)}\boldsymbol{h}^{(l-1)} + \boldsymbol{b}^{(l)}\big)$
  for $l = 1, \ldots, L-1$, starting from $\boldsymbol{h}^{(0)} = \boldsymbol{x}$.
- The last layer is the softmax:
  $\boldsymbol{p} = \mathrm{softmax}\big(\boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)}\big)$
- Many hidden layers: a **deep** network, hence "deep learning".



WHY HIDDEN LAYERS?

Logistic regression can't learn XOR

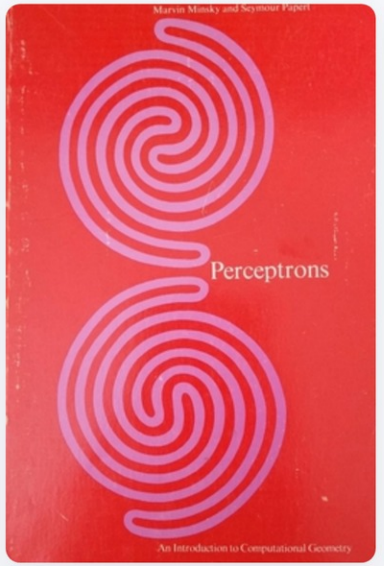


HISTORY

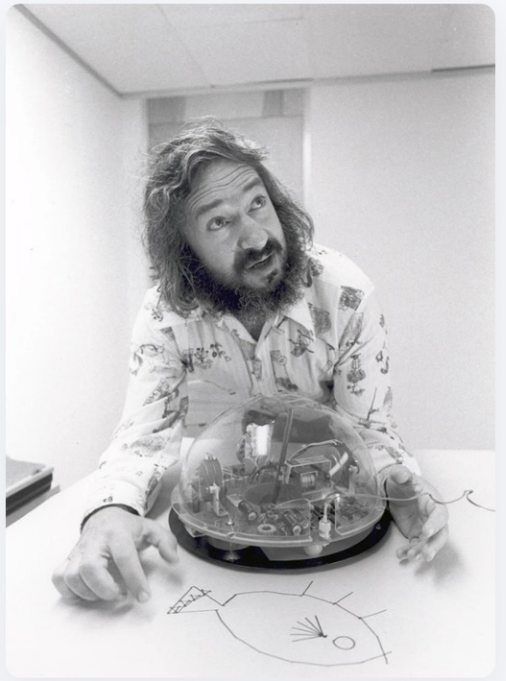
Perceptrons (1969)



Marvin Minsky



Perceptrons, MIT Press, 1969



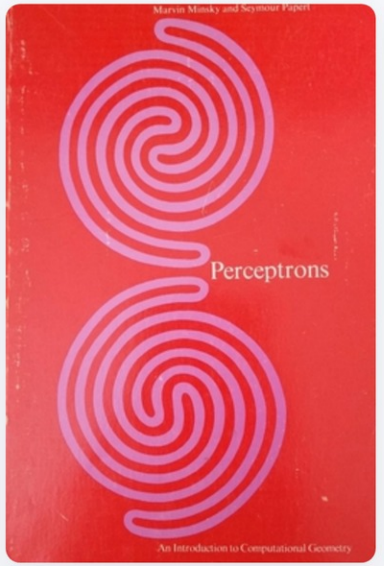
Seymour Papert

HISTORY

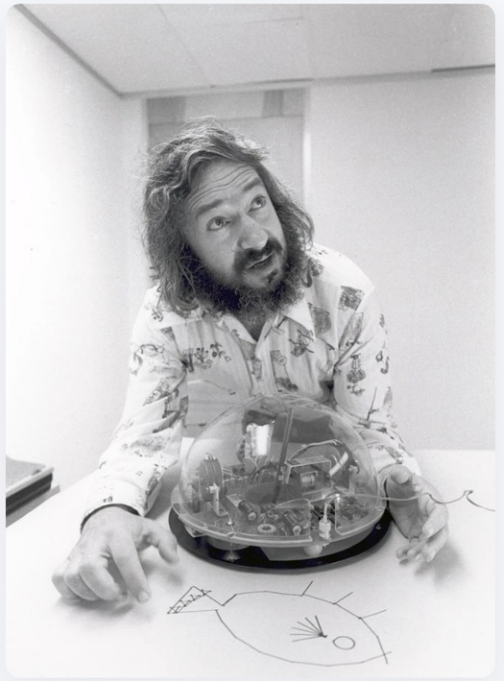
Perceptrons (1969)



Marvin Minsky



Perceptrons, MIT Press, 1969



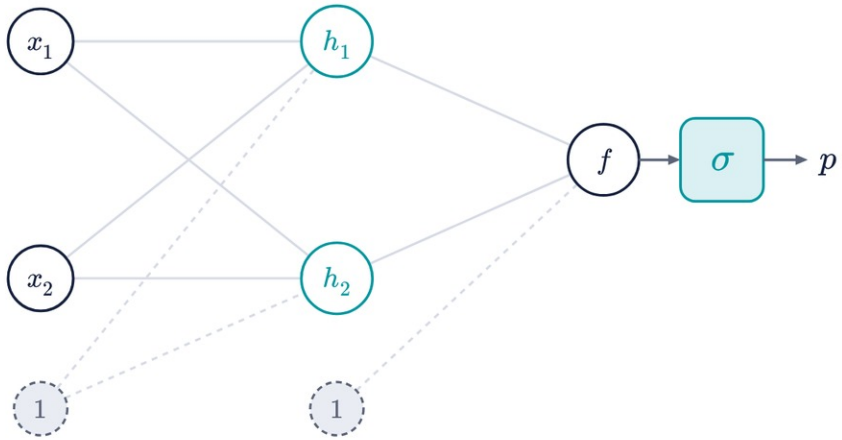
Seymour Papert

They showed limitations of single-layer perceptrons. There are things they cannot compute, such as XOR.

CHECK IT BY HAND

XOR with two hidden units

- Inputs 0 or 1.

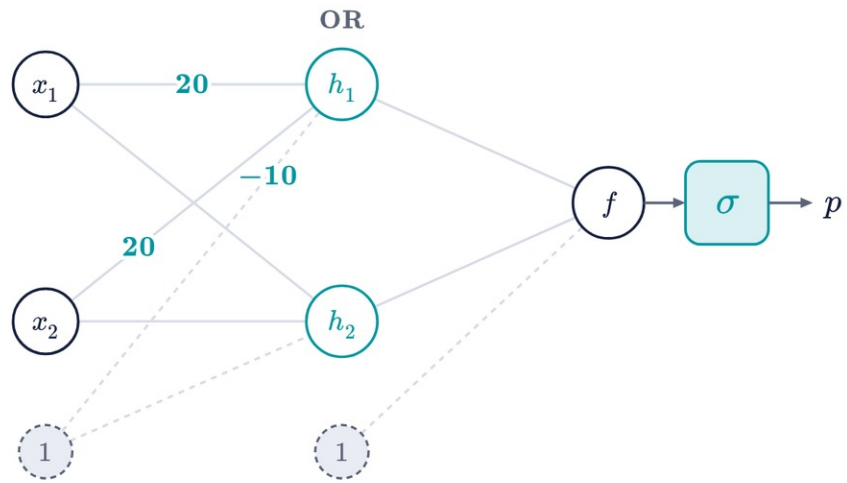


x_1	x_2
0	0
0	1
1	0
1	1

CHECK IT BY HAND

XOR with two hidden units

- Inputs 0 or 1.
- $h_1 = \sigma(20x_1 + 20x_2 - 10)$: about 1 if either input is 1. That is OR.

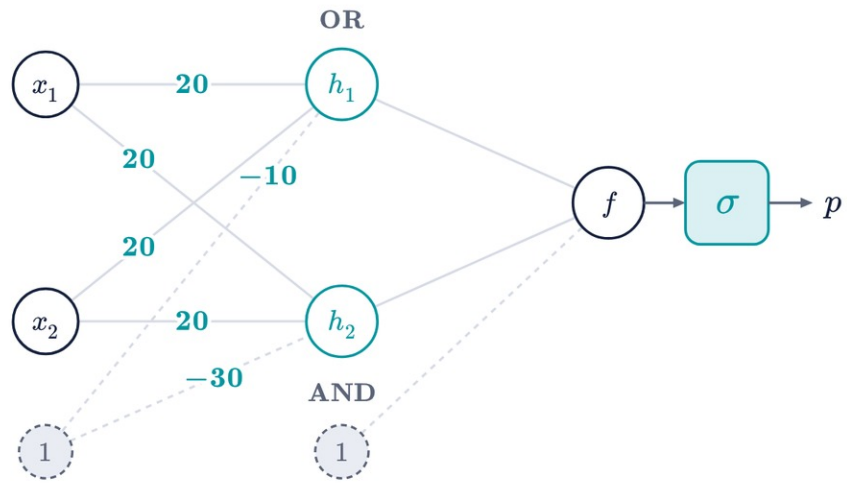


x_1	x_2	h_1 (OR)
0	0	
0	1	
1	0	
1	1	

CHECK IT BY HAND

XOR with two hidden units

- Inputs 0 or 1.
- $h_1 = \sigma(20x_1 + 20x_2 - 10)$: about 1 if either input is 1. That is OR.
- $h_2 = \sigma(20x_1 + 20x_2 - 30)$: about 1 only if both are. That is AND.

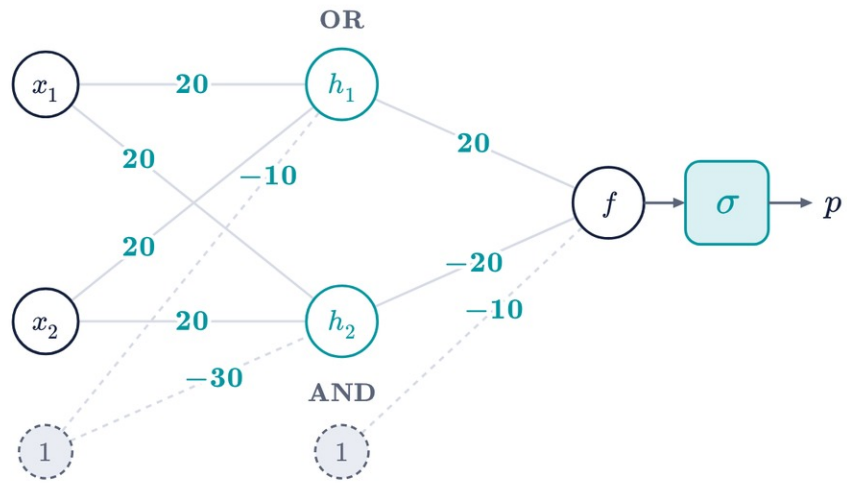


x_1	x_2	h_1 (OR)	h_2 (AND)
0	0		
0	1		
1	0		
1	1		

CHECK IT BY HAND

XOR with two hidden units

- Inputs 0 or 1.
- $h_1 = \sigma(20x_1 + 20x_2 - 10)$: about 1 if either input is 1. That is OR.
- $h_2 = \sigma(20x_1 + 20x_2 - 30)$: about 1 only if both are. That is AND.
- $f = 20h_1 - 20h_2 - 10$, $p = \sigma(f)$: OR and not AND. That is XOR.

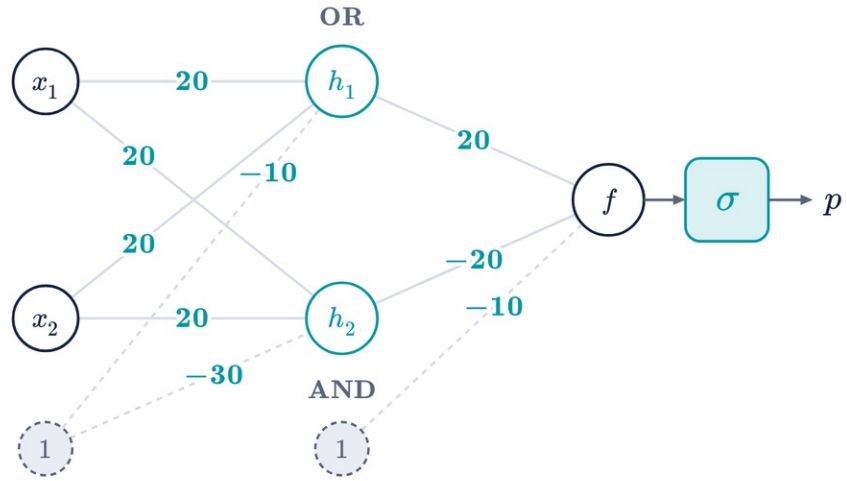


x_1	x_2	h_1 (OR)	h_2 (AND)	f	p
0	0				
0	1				
1	0				
1	1				

CHECK IT BY HAND

XOR with two hidden units

- Inputs 0 or 1.
- $h_1 = \sigma(20x_1 + 20x_2 - 10)$: about 1 if either input is 1. That is OR.
- $h_2 = \sigma(20x_1 + 20x_2 - 30)$: about 1 only if both are. That is AND.
- $f = 20h_1 - 20h_2 - 10$, $p = \sigma(f)$: OR and not AND. That is XOR.



x_1	x_2	h_1 (OR)	h_2 (AND)	f	p
0	0	0.00	0.00	−10.0	**0.00**
0	1	1.00	0.00	10.0	**1.00**
1	0	1.00	0.00	10.0	**1.00**
1	1	1.00	1.00	−10.0	**0.00**

TRAINING

How do we choose the weights?

- For XOR we set them by hand. Real networks have thousands or millions of weights.

TRAINING

How do we choose the weights?

- For XOR we set them by hand. Real networks have thousands or millions of weights.
- Choose the weights that make a **loss** on the training data small.

TRAINING

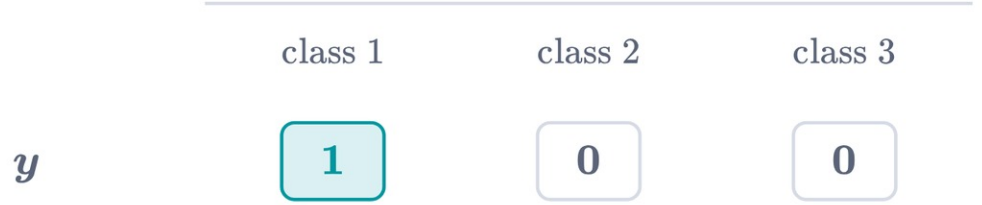
How do we choose the weights?

- For XOR we set them by hand. Real networks have thousands or millions of weights.
- Choose the weights that make a **loss** on the training data small.
- Two questions: which loss, and how to make it small.

THE LOSS

Cross-entropy

- Write the label as a **one-hot** vector, with a 1 for the true class: $\boldsymbol{y} = (1, 0, 0)$ for class 1.



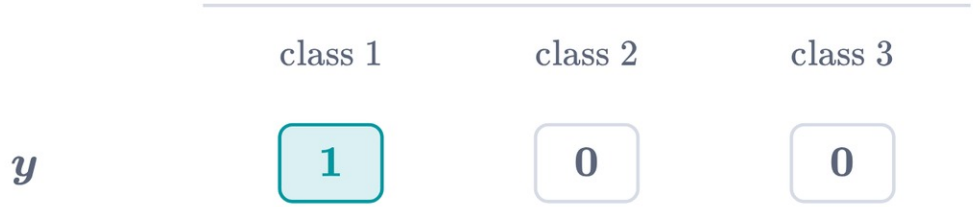
THE LOSS

Cross-entropy

- Write the label as a **one-hot** vector, with a 1 for the true class: $\boldsymbol{y} = (1, 0, 0)$ for class 1.
- The cross-entropy of one point:

  $$\mathcal{L} = -\sum_k y_k \log p_k = -\log p(\text{true class})$$

  Add it up over the training data.



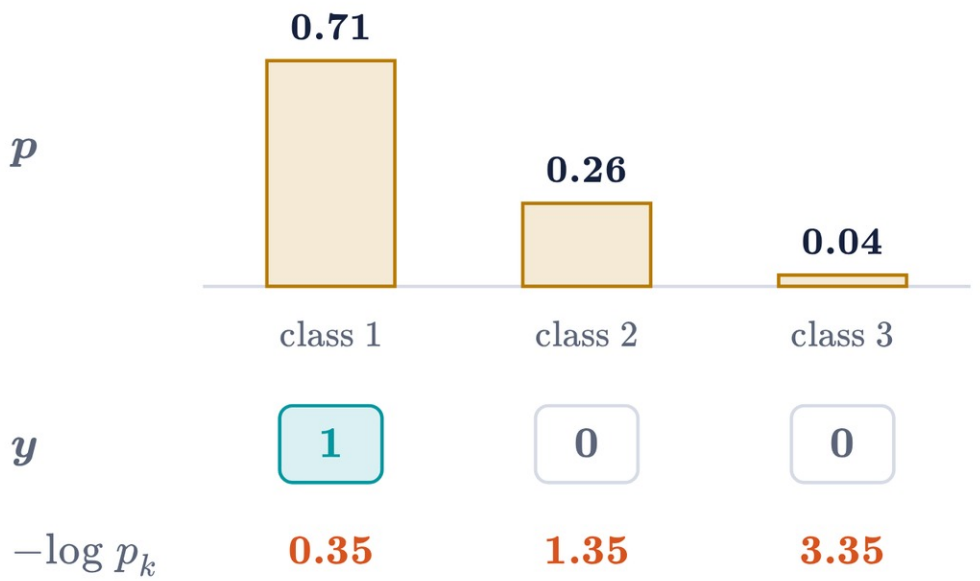
THE LOSS

Cross-entropy

- Write the label as a **one-hot** vector, with a 1 for the true class: $\boldsymbol{y} = (1, 0, 0)$ for class 1.
- The cross-entropy of one point:

  $$\mathcal{L} = -\sum_k y_k \log p_k = -\log p(\text{true class})$$

  Add it up over the training data.
- With $\boldsymbol{p} = (0.71, 0.26, 0.04)$: if the true class is 1, the loss is 0.35; if it is 3, the loss is 3.35.



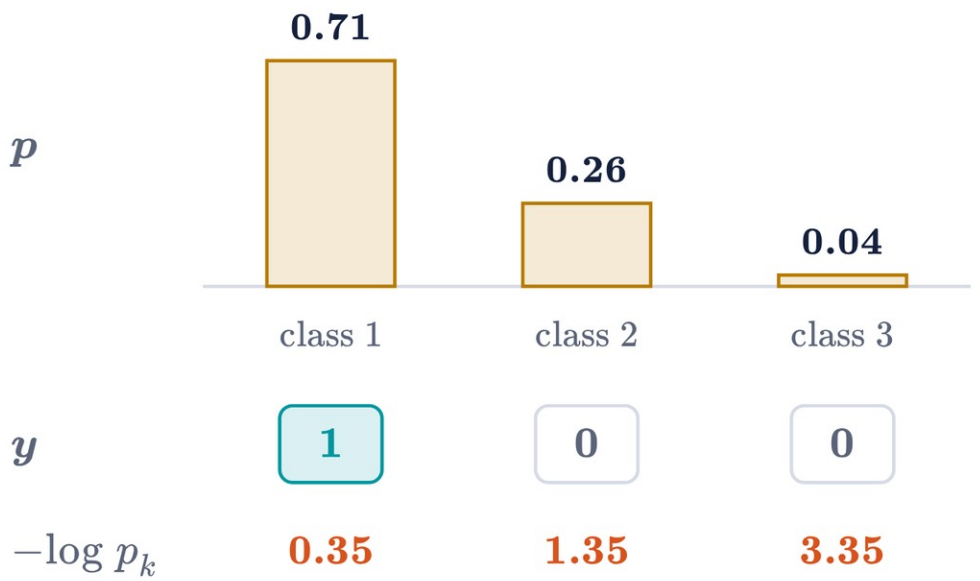
THE LOSS

Cross-entropy

- Write the label as a **one-hot** vector, with a 1 for the true class: $\boldsymbol{y} = (1, 0, 0)$ for class 1.
- The cross-entropy of one point:

  $$\mathcal{L} = -\sum_k y_k \log p_k = -\log p(\text{true class})$$

  Add it up over the training data.
- With $\boldsymbol{p} = (0.71, 0.26, 0.04)$: if the true class is 1, the loss is 0.35; if it is 3, the loss is 3.35.
- With two classes, it is last lecture's log loss.



PUTTING IT TOGETHER

Multilayer perceptrons

Model

$$\boldsymbol{h}^{(0)} = \boldsymbol{x}, \quad \boldsymbol{h}^{(l)} = \varphi\big(\boldsymbol{W}^{(l)}\boldsymbol{h}^{(l-1)} + \boldsymbol{b}^{(l)}\big)$$

$$\boldsymbol{p} = \mathrm{softmax}\big(\boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)}\big)$$

φ is the activation function. The parameters $\boldsymbol{\theta}$ are all the weights and biases.

PUTTING IT TOGETHER

Multilayer perceptrons

Model

$$\boldsymbol{h}^{(0)} = \boldsymbol{x}, \quad \boldsymbol{h}^{(l)} = \varphi\big(\boldsymbol{W}^{(l)}\boldsymbol{h}^{(l-1)} + \boldsymbol{b}^{(l)}\big)$$

$$\boldsymbol{p} = \mathrm{softmax}\big(\boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)}\big)$$

φ is the activation function. The parameters $\boldsymbol{\theta}$ are all the weights and biases.

Loss

$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_{i} \log p(\text{true class of } \boldsymbol{x}_i)$$

Cross-entropy, added up over the training data.

PUTTING IT TOGETHER

Multilayer perceptrons

Model

$$\boldsymbol{h}^{(0)} = \boldsymbol{x}, \quad \boldsymbol{h}^{(l)} = \varphi\big(\boldsymbol{W}^{(l)}\boldsymbol{h}^{(l-1)} + \boldsymbol{b}^{(l)}\big)$$

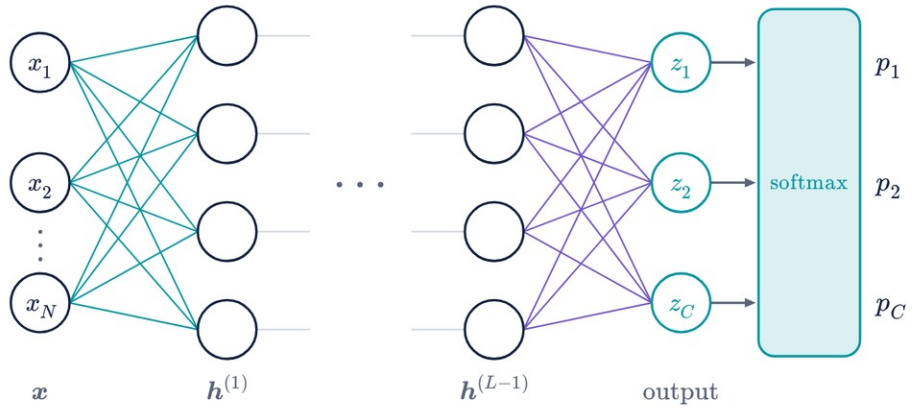
$$\boldsymbol{p} = \mathrm{softmax}\big(\boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)}\big)$$

φ is the activation function. The parameters $\boldsymbol{\theta}$ are all the weights and biases.

Loss

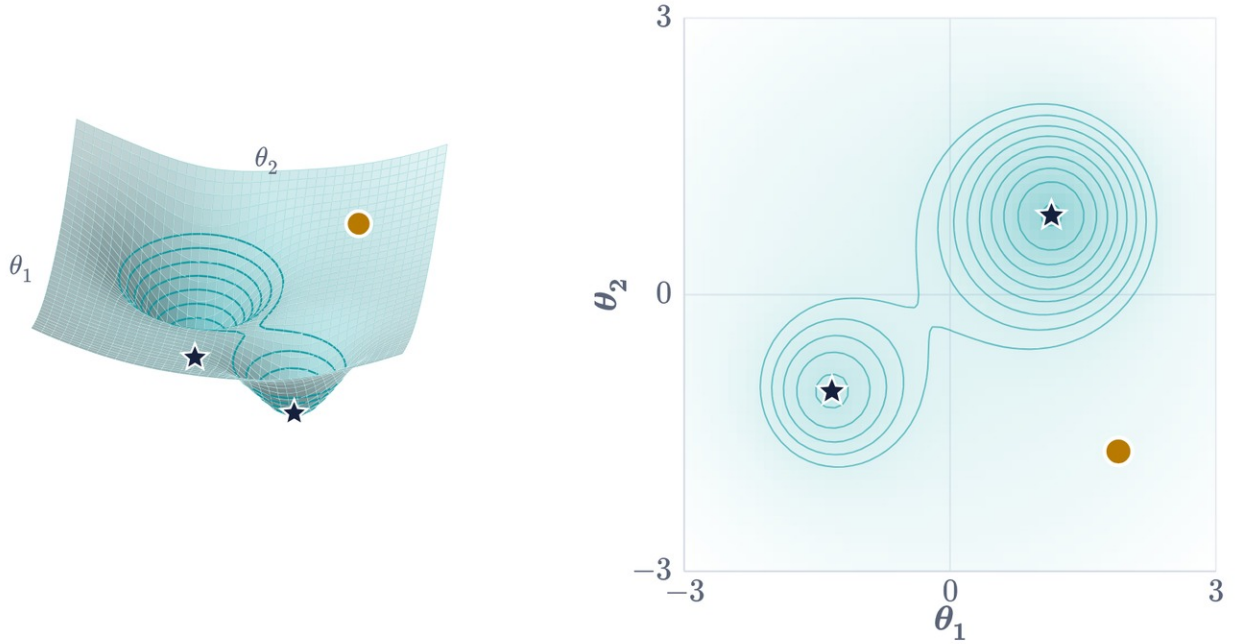
$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_i \log p(\text{true class of } \boldsymbol{x}_i)$$

Cross-entropy, added up over the training data.



The loss landscape

- The loss depends on the parameters $\boldsymbol{\theta}$. Picture it as a surface over $\boldsymbol{\theta}$.



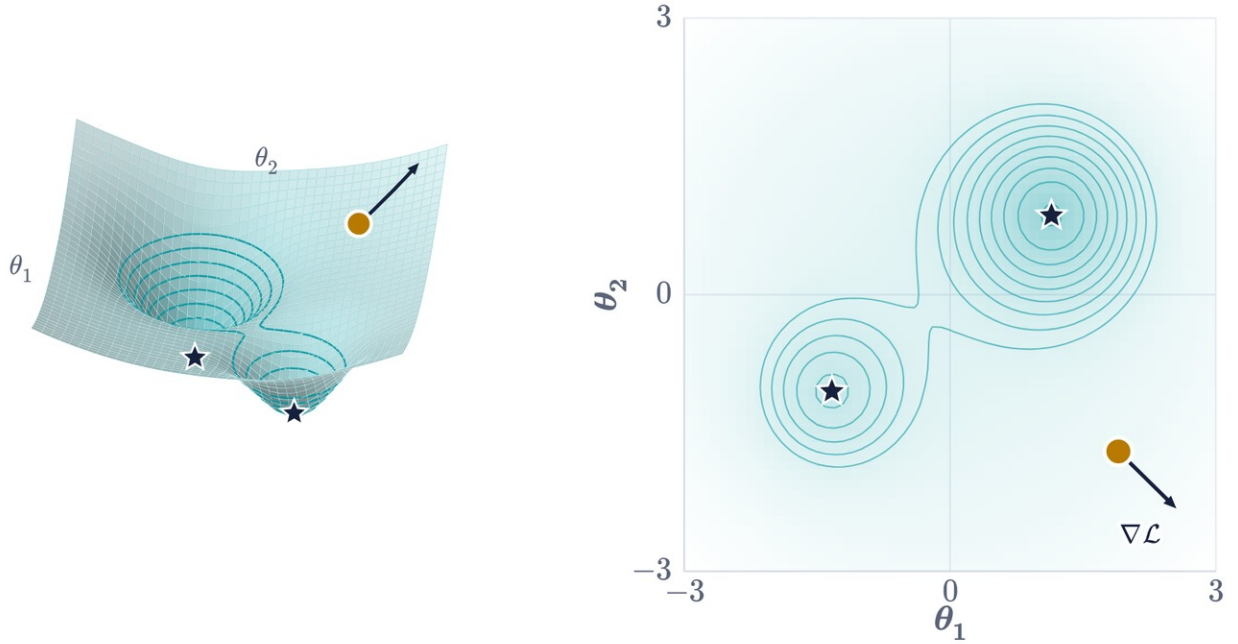
A made-up loss over two parameters. Left: the height is the loss. Right: the same surface seen from above.

The loss landscape

- The loss depends on the parameters $\boldsymbol{\theta}$. Picture it as a surface over $\boldsymbol{\theta}$.
- The gradient is the vector of partial derivatives:

  $$\nabla \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2} \right)$$

  It points uphill, the steepest way up.



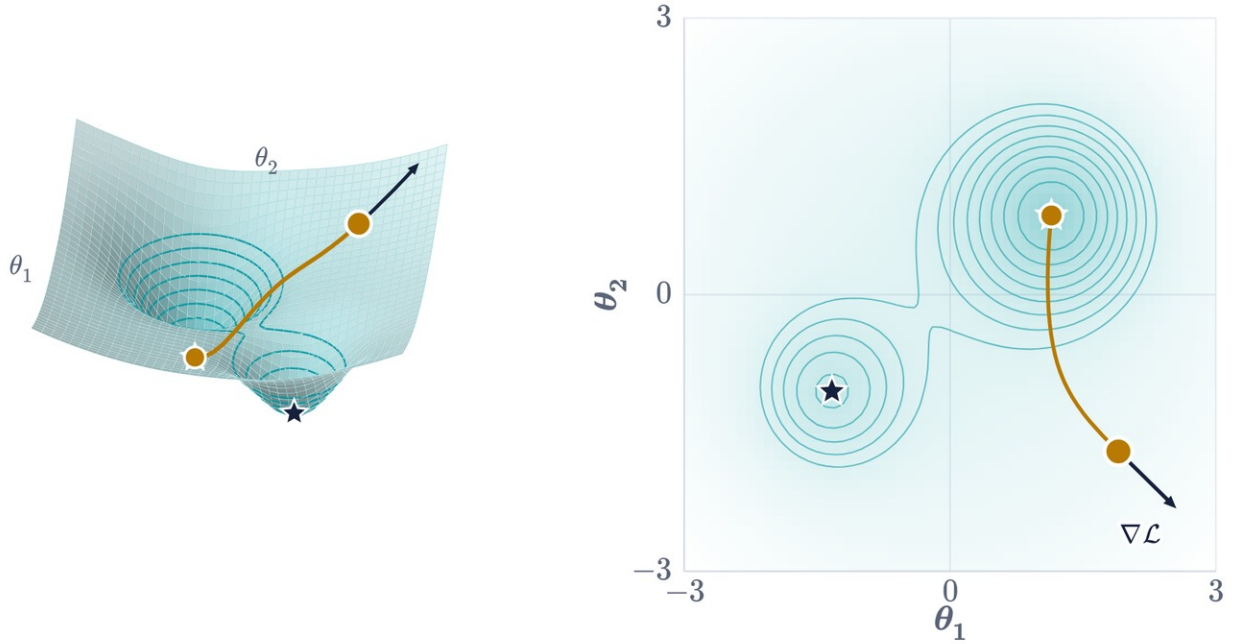
A made-up loss over two parameters. Left: the height is the loss. Right: the same surface seen from above.

The loss landscape

- The loss depends on the parameters $\boldsymbol{\theta}$. Picture it as a surface over $\boldsymbol{\theta}$.
- The gradient is the vector of partial derivatives:

  $$\nabla \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2} \right)$$

  It points uphill, the steepest way up.
- Step the other way, again and again: **gradient descent**.



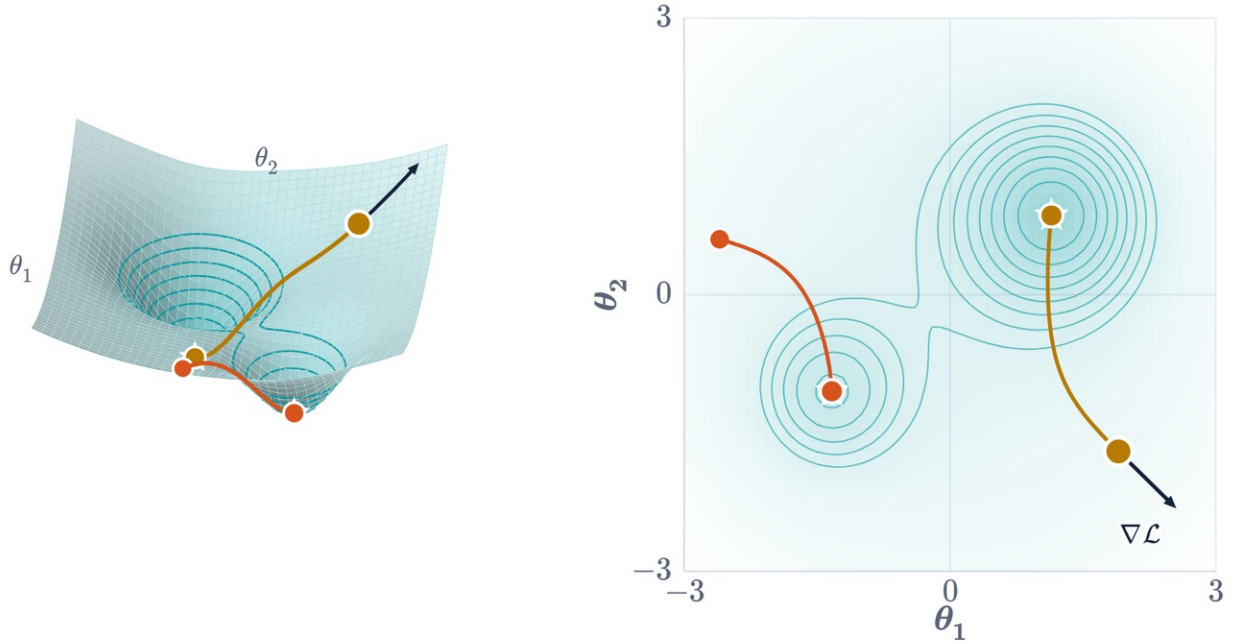
A made-up loss over two parameters. Left: the height is the loss. Right: the same surface seen from above.

The loss landscape

- The loss depends on the parameters $\boldsymbol{\theta}$. Picture it as a surface over $\boldsymbol{\theta}$.
- The gradient is the vector of partial derivatives:

  $$\nabla \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2} \right)$$

  It points uphill, the steepest way up.
- Step the other way, again and again: **gradient descent**.
- A different start can end in a different minimum.



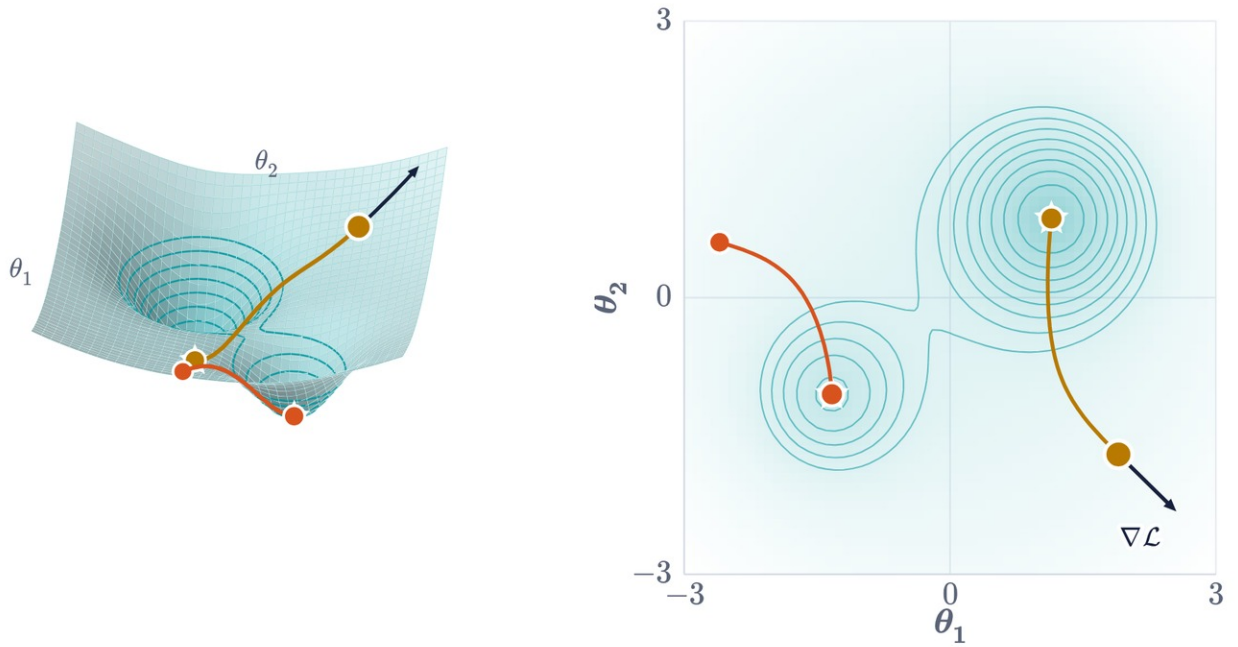
A made-up loss over two parameters. Left: the height is the loss. Right: the same surface seen from above.

The loss landscape

- The loss depends on the parameters $\boldsymbol{\theta}$. Picture it as a surface over $\boldsymbol{\theta}$.
- The gradient is the vector of partial derivatives:

  $$\nabla \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2} \right)$$

  It points uphill, the steepest way up.
- Step the other way, again and again: **gradient descent**.
- A different start can end in a different minimum.
- Logistic regression's loss is a single bowl: one minimum. A network's loss is not.



A made-up loss over two parameters. Left: the height is the loss. Right: the same surface seen from above.

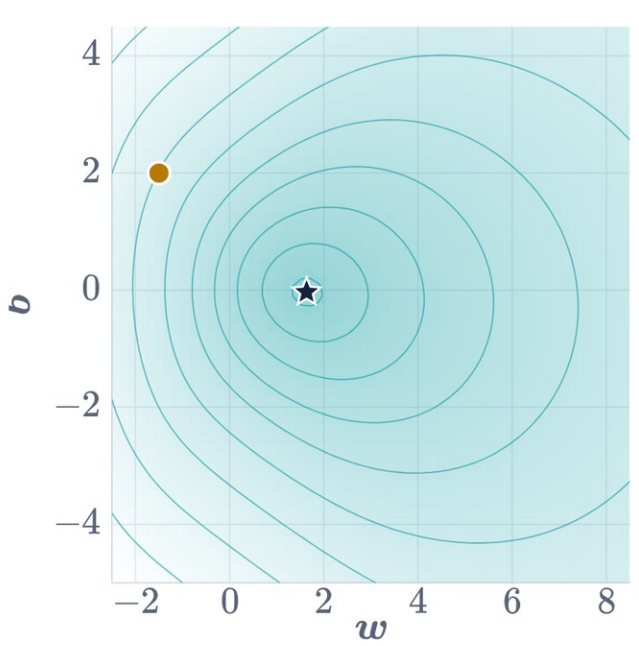
HOW TO TRAIN IT?

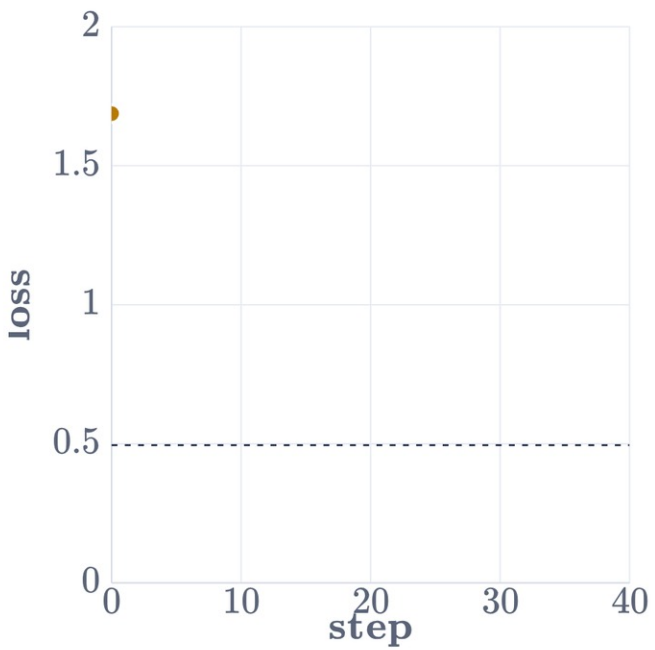
Gradient descent

- Update all parameters at once:

  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}(\boldsymbol{\theta})$

  η is the **learning rate**.





η	STEP	LOSS
0.1	0	1.687

$\eta = 0.1$ $\eta = 1$ $\eta = 20$ ▶

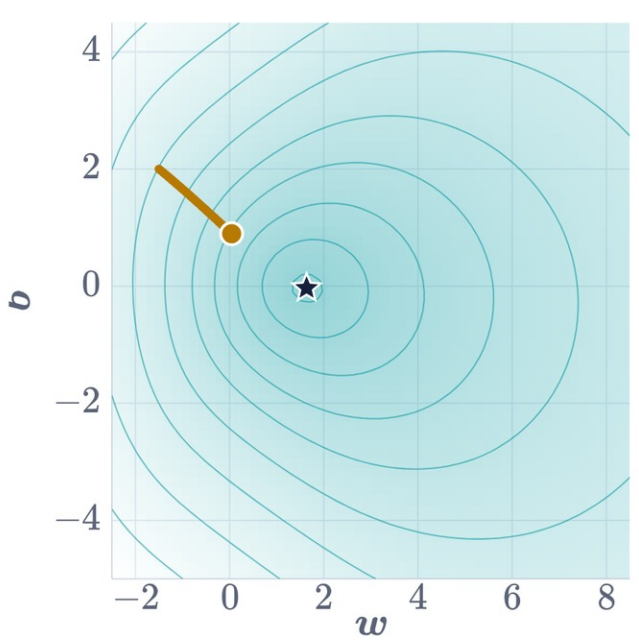
HOW TO TRAIN IT?

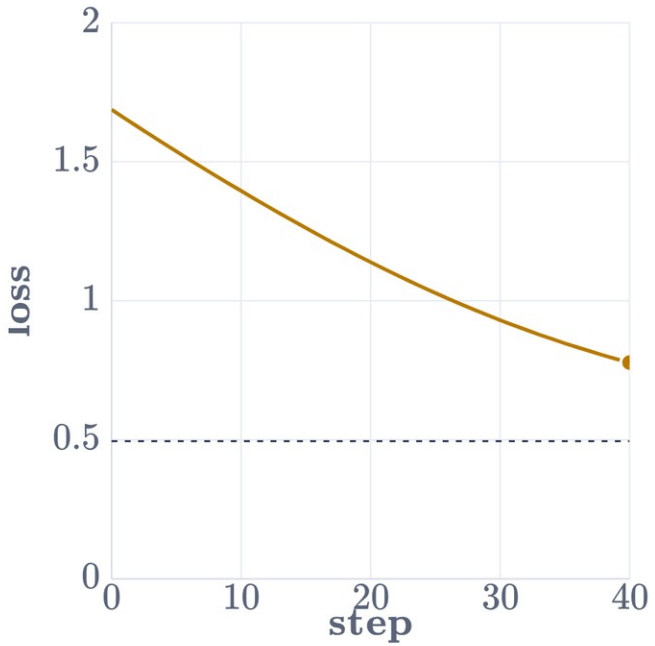
Gradient descent

- Update all parameters at once:

  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}(\boldsymbol{\theta})$

  η is the **learning rate**.
- Too small: many tiny steps.





η	STEP	LOSS
0.1	40	0.778

$\eta = 0.1$ $\eta = 1$ $\eta = 20$ ▶

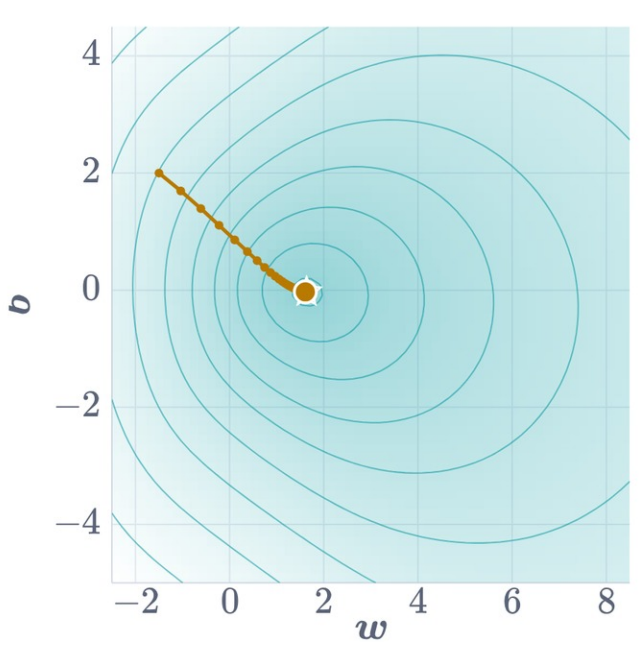
HOW TO TRAIN IT?

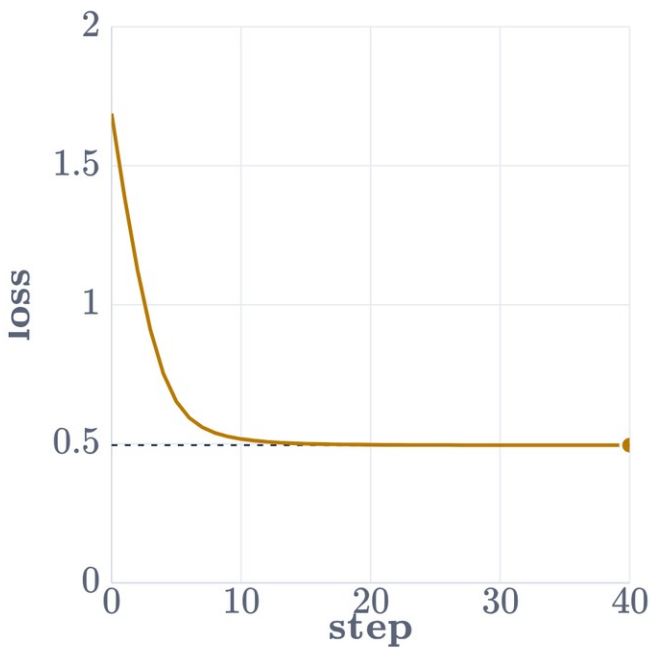
Gradient descent

- Update all parameters at once:

  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}(\boldsymbol{\theta})$

  η is the **learning rate**.
- Too small: many tiny steps.
- About right: a few steps to the minimum.





η	STEP	LOSS
1	40	0.495

$\eta = 0.1$ $\eta = 1$ $\eta = 20$ ▶

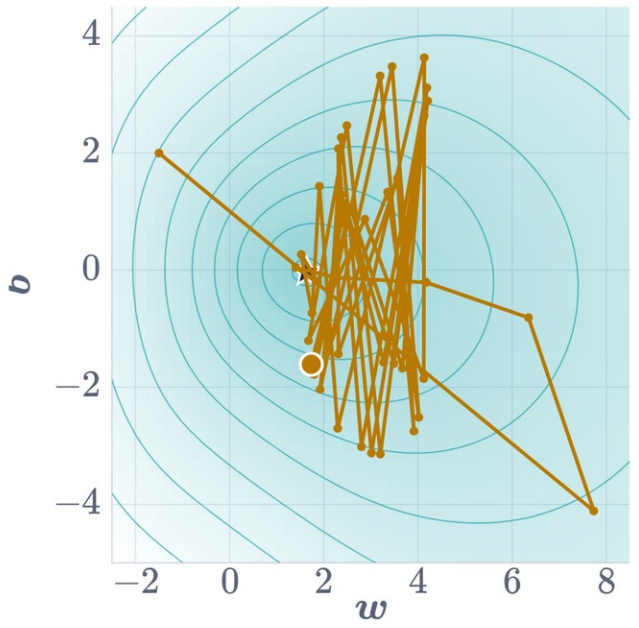
HOW TO TRAIN IT?

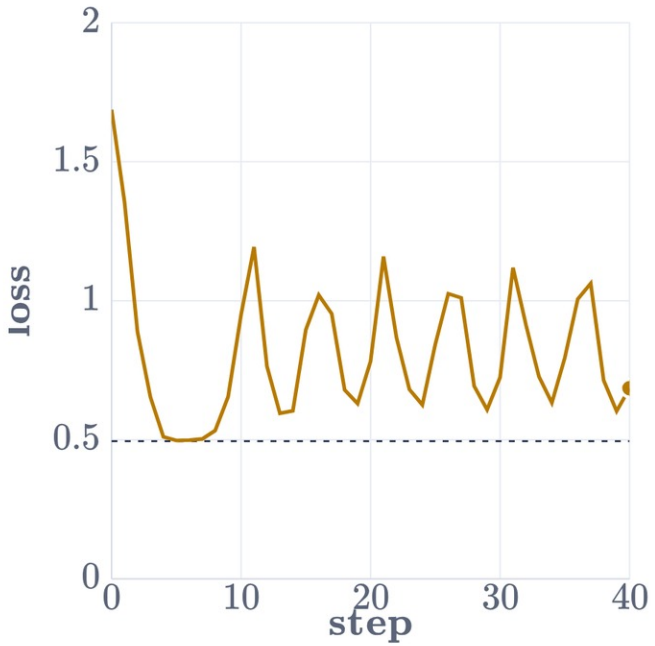
Gradient descent

- Update all parameters at once:

  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}(\boldsymbol{\theta})$

  η is the **learning rate**.
- Too small: many tiny steps.
- About right: a few steps to the minimum.
- Too large: it overshoots and the loss goes back up.





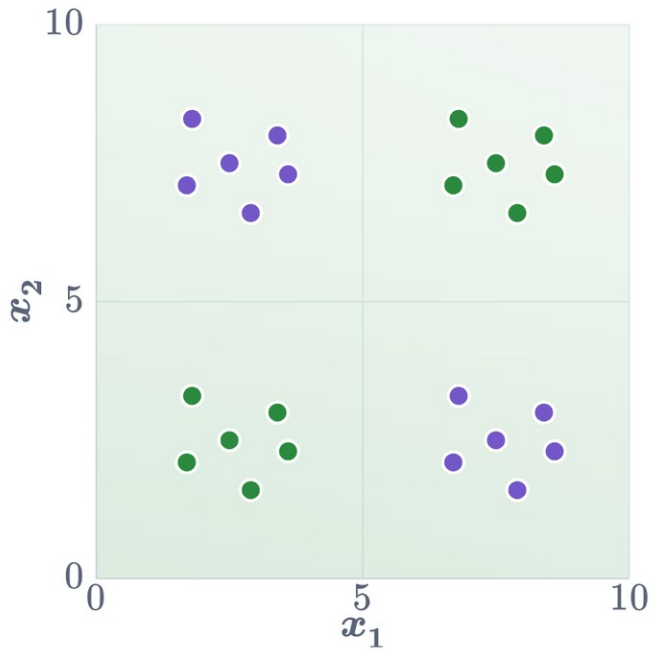
η	STEP	LOSS
20	40	0.686

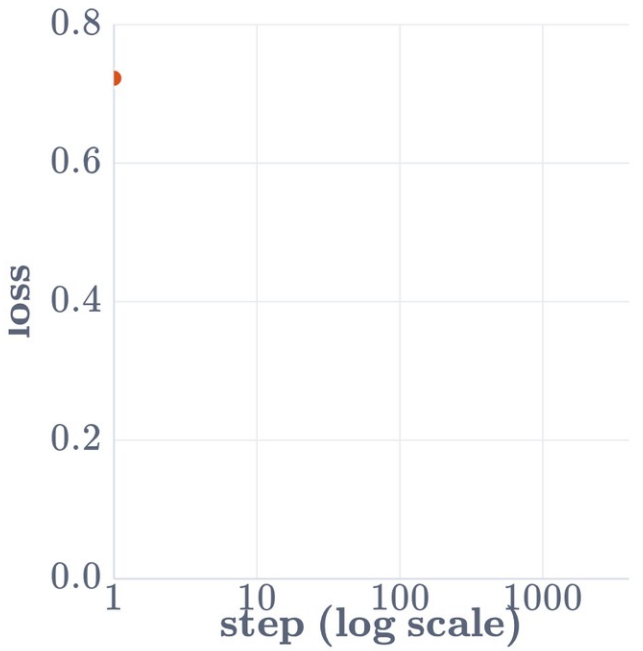
$\eta = 0.1$ $\eta = 1$ $\eta = 20$ ▶

GRADIENT DESCENT ON A NETWORK

Training learns XOR

- Three sigmoid hidden units, two softmax outputs, random starting weights, gradient descent on the cross-entropy.





STEP	LOSS	ACCURACY
0	0.7227	50%

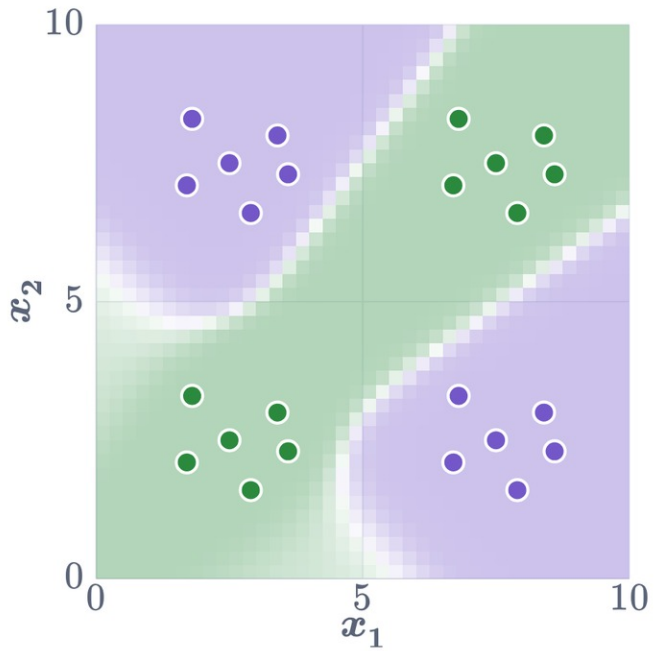
▶ step

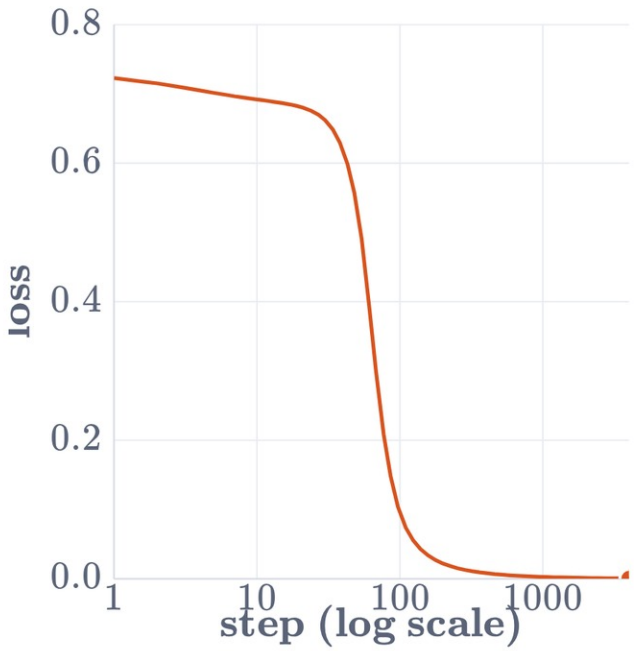
GRADIENT DESCENT ON A NETWORK

Training learns XOR

- Three sigmoid hidden units, two softmax outputs, random starting weights, gradient descent on the cross-entropy.
- The decision region bends until every point is on the right side.

How to compute the gradient for every weight: next class.





STEP	LOSS	ACCURACY
4000	0.0006	100%

▶ step

SUMMARY

Takeaways

Structure
Neurons in layers, each applying an activation function such as the sigmoid or ReLU. Hidden layers learn features, so XOR becomes possible.

Softmax
One score per class, turned into probabilities. With two classes it is the sigmoid.

Loss
Cross-entropy, $-\log p(\text{true class})$. Confident and wrong costs a lot.

Training
Gradient descent, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}$. The learning rate sets the step size.

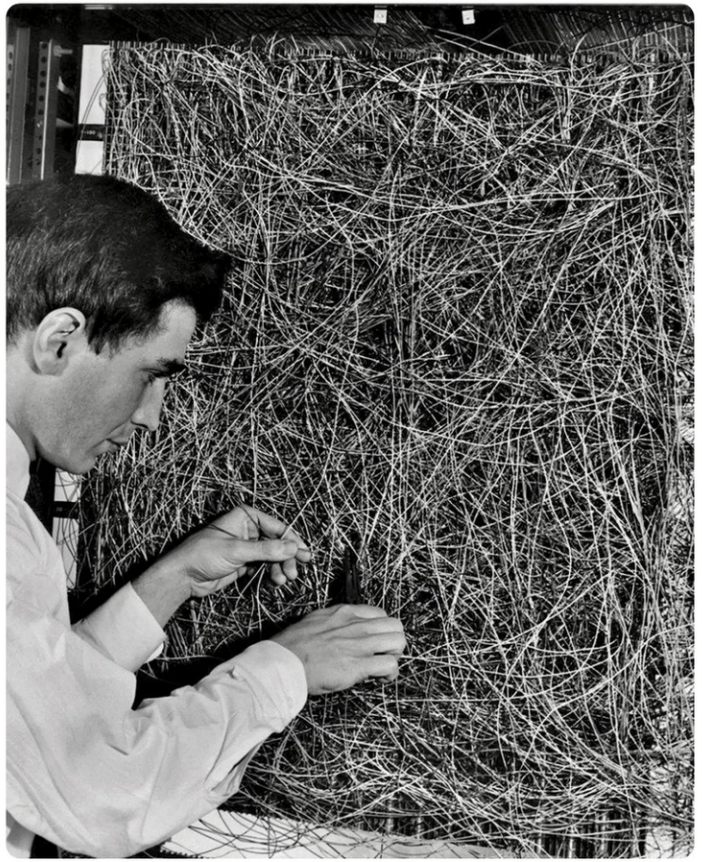
Thank you.

Questions?

BONUS

The perceptron (1958)

- Rosenblatt's perceptron uses the same score as logistic regression, $f(\boldsymbol{x}) = \boldsymbol{w} \cdot \boldsymbol{x} + b$, but without the sigmoid.

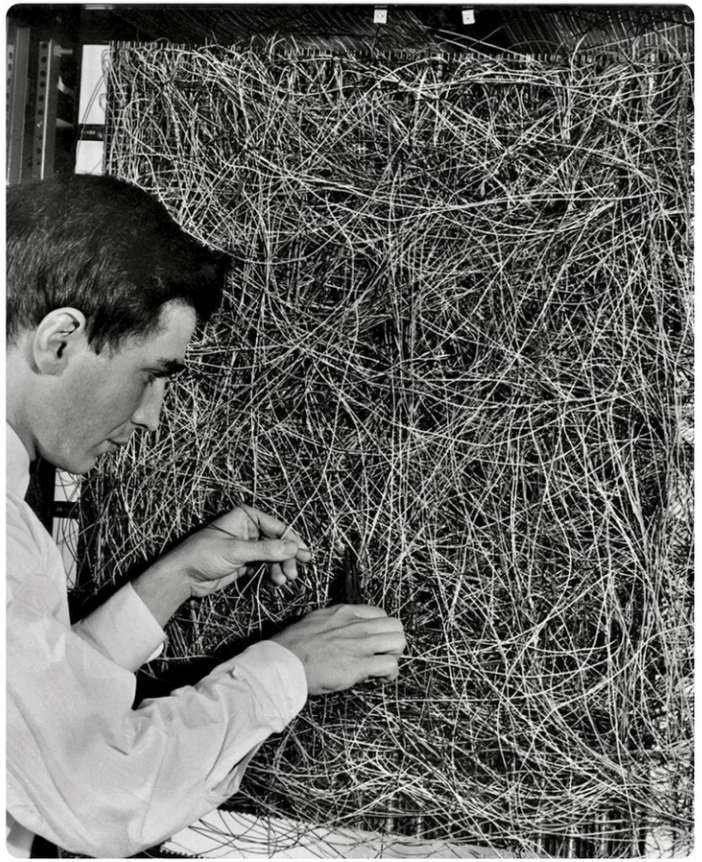


Frank Rosenblatt wiring the Mark I Perceptron, Cornell Aeronautical Laboratory.

BONUS

The perceptron (1958)

- Rosenblatt's perceptron uses the same score as logistic regression, $f(\boldsymbol{x}) = \boldsymbol{w} \cdot \boldsymbol{x} + b$, but without the sigmoid.
- A step instead of the sigmoid: predict the sign of $f(\boldsymbol{x})$.



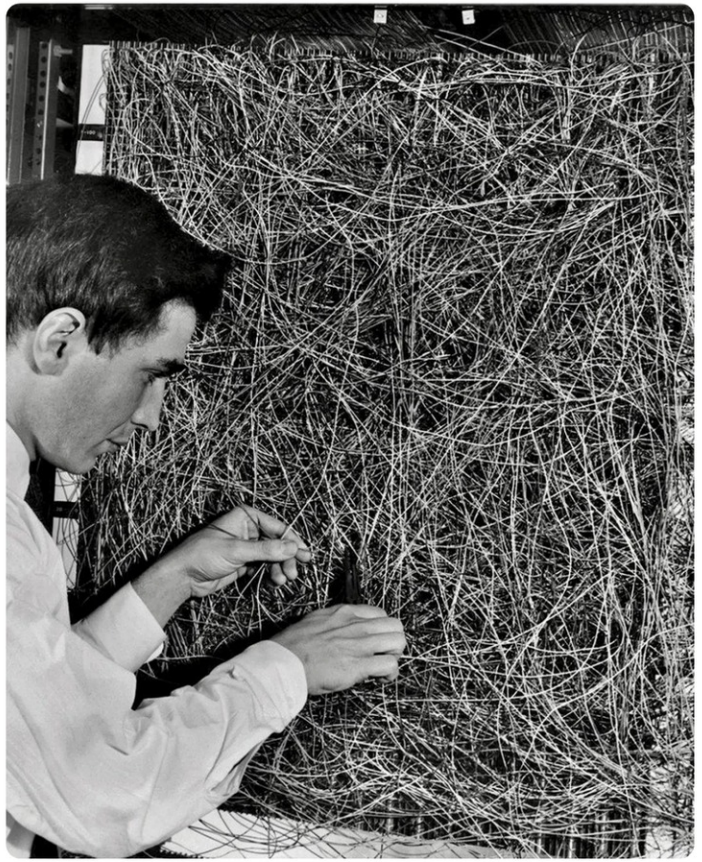
Frank Rosenblatt wiring the Mark I Perceptron, Cornell Aeronautical Laboratory.

BONUS

The perceptron (1958)

- Rosenblatt's perceptron uses the same score as logistic regression, $f(\boldsymbol{x}) = \boldsymbol{w} \cdot \boldsymbol{x} + b$, but without the sigmoid.
- A step instead of the sigmoid: predict the sign of $f(\boldsymbol{x})$.
- Its learning rule, on a mistake:

  $\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta\, y_i \boldsymbol{x}_i, \quad b \leftarrow b + \eta\, y_i$



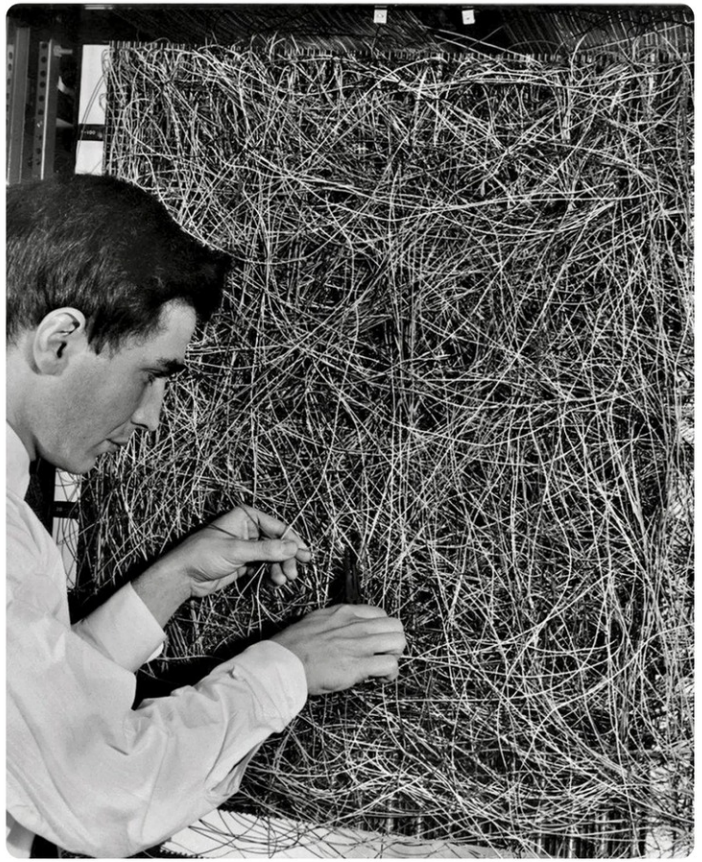
Frank Rosenblatt wiring the Mark I Perceptron, Cornell Aeronautical Laboratory.

BONUS

The perceptron (1958)

- Rosenblatt's perceptron uses the same score as logistic regression, $f(\boldsymbol{x}) = \boldsymbol{w} \cdot \boldsymbol{x} + b$, but without the sigmoid.
- A step instead of the sigmoid: predict the sign of $f(\boldsymbol{x})$.
- Its learning rule, on a mistake:

  $\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta\, y_i \boldsymbol{x}_i, \quad b \leftarrow b + \eta\, y_i$
- That is gradient descent, one point at a time, on the loss $\max(0, -y\, f(\boldsymbol{x}))$.

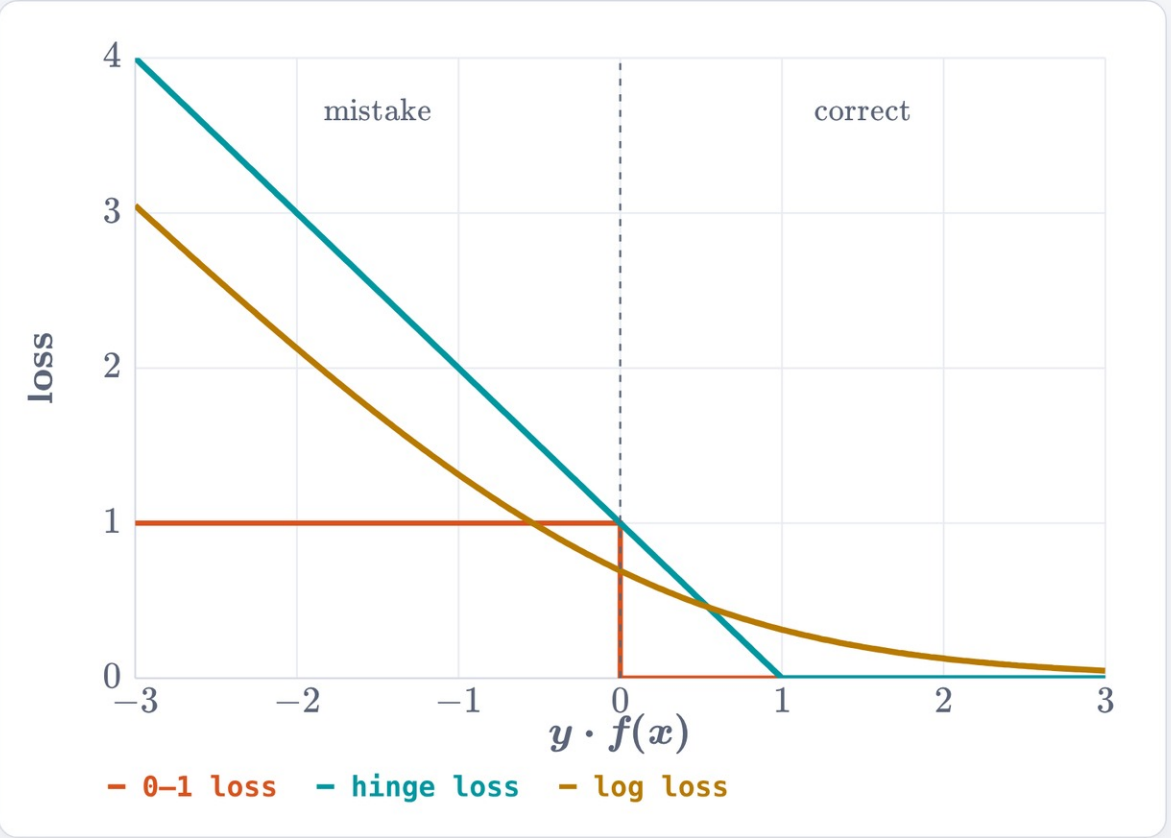


Frank Rosenblatt wiring the Mark I Perceptron, Cornell Aeronautical Laboratory.

BONUS

Losses compared

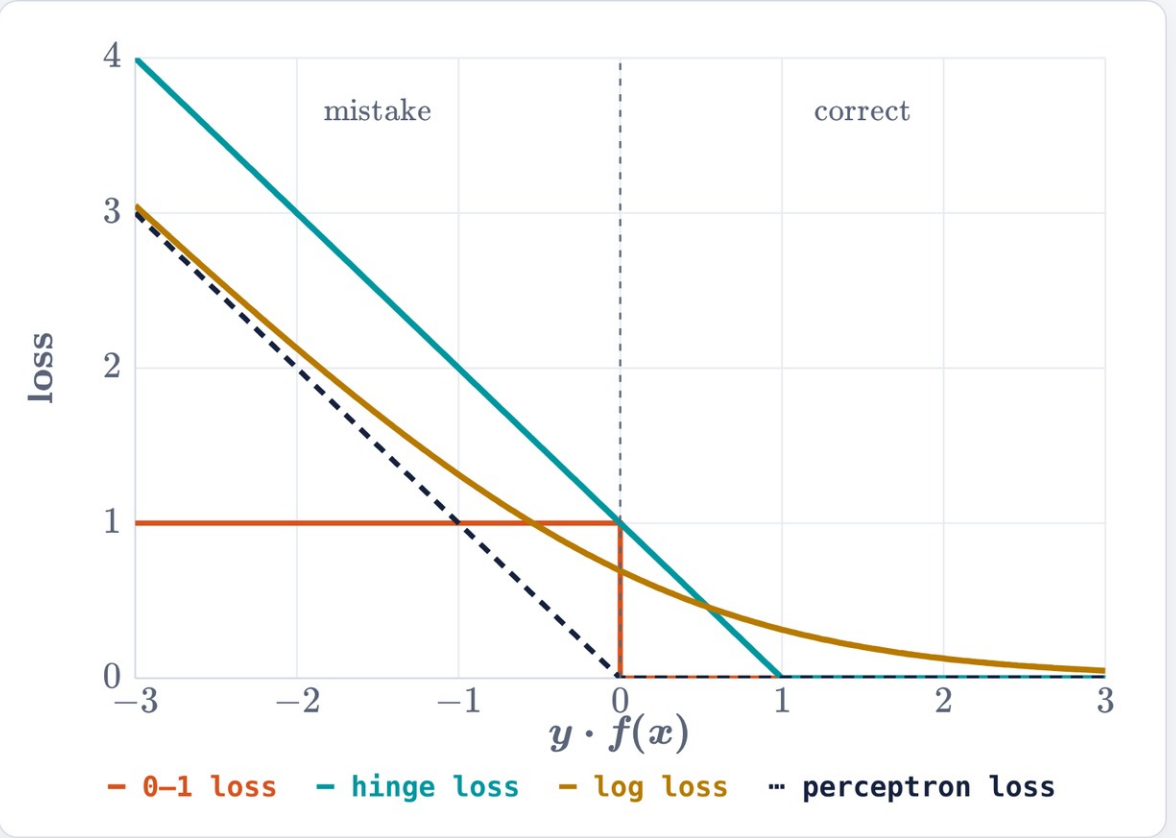
- All four depend only on the signed score $y\,f(\boldsymbol{x})$: right of 0 is correct, left of 0 is a mistake.



BONUS

Losses compared

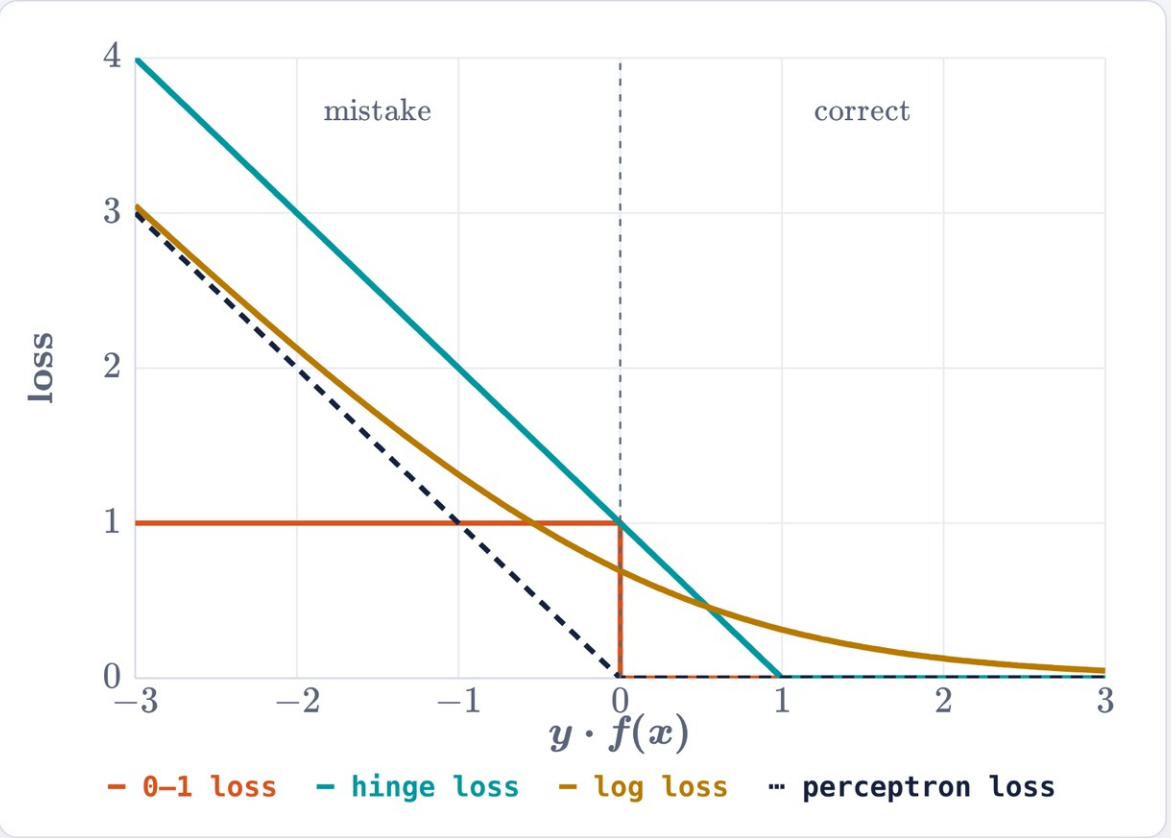
- All four depend only on the signed score $y\,f(\boldsymbol{x})$: right of 0 is correct, left of 0 is a mistake.
- **Perceptron loss**: $\max\big(0, -y\,f(\boldsymbol{x})\big)$. Zero for every correct point, however close to the boundary.



BONUS

Losses compared

- All four depend only on the signed score $y\,f(\boldsymbol{x})$: right of 0 is correct, left of 0 is a mistake.
- **Perceptron loss**: $\max\big(0, -y\,f(\boldsymbol{x})\big)$. Zero for every correct point, however close to the boundary.
- The hinge loss is the perceptron loss shifted right by 1: a point must be correct by a margin to cost nothing.



BONUS

Softmax vs argmax

- The **argmax** puts all the probability on the top score: $(1, 0, 0)$ for $\boldsymbol{z} = (2, 1, -1)$. It jumps when two scores swap.

BONUS

Softmax vs argmax

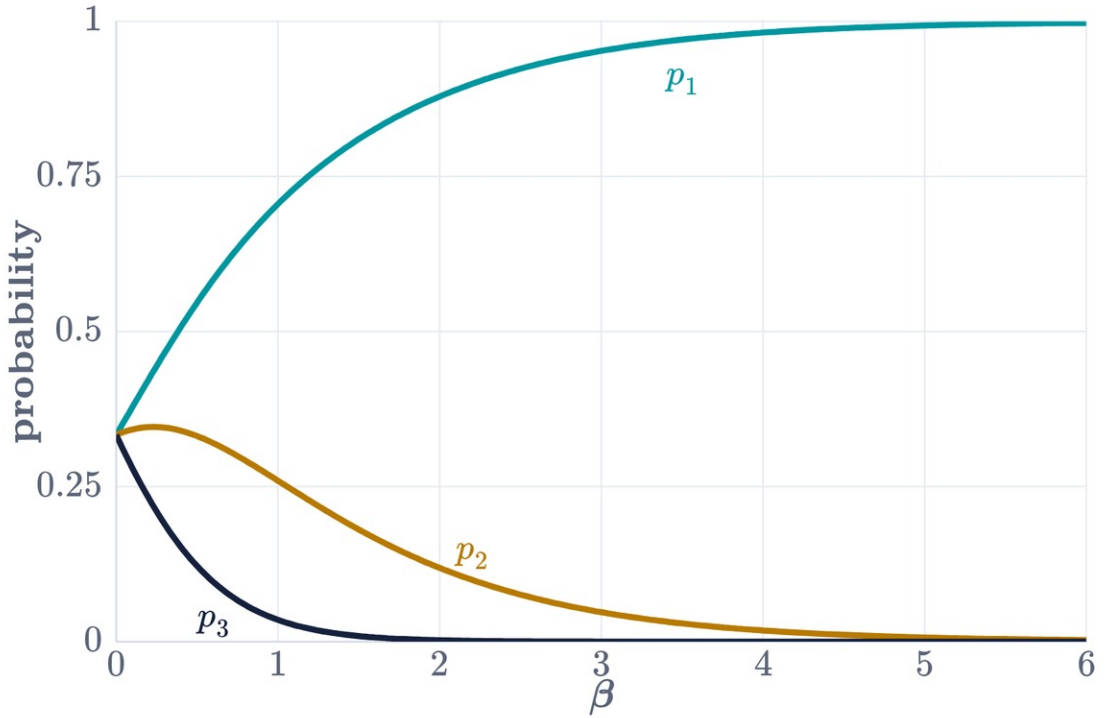
- The **argmax** puts all the probability on the top score: $(1, 0, 0)$ for $\boldsymbol{z} = (2, 1, -1)$. It jumps when two scores swap.
- Dividing by the sum also gives numbers that add to 1, but only for positive scores, and it keeps the ratios: $(3, 2, 1)$ gives $(0.50, 0.33, 0.17)$. The softmax gives $(0.67, 0.24, 0.09)$: the exponential stretches the gaps.

BONUS

Softmax vs argmax

- The **argmax** puts all the probability on the top score: $(1, 0, 0)$ for $\boldsymbol{z} = (2, 1, -1)$. It jumps when two scores swap.
- Dividing by the sum also gives numbers that add to 1, but only for positive scores, and it keeps the ratios: $(3, 2, 1)$ gives $(0.50, 0.33, 0.17)$. The softmax gives $(0.67, 0.24, 0.09)$: the exponential stretches the gaps.
- Put a coefficient β in front of the scores:

$$p_k = \frac{e^{\beta z_k}}{\sum_j e^{\beta z_j}}$$



The scores $\boldsymbol{z} = (2, 1, -1)$ from the softmax slide.

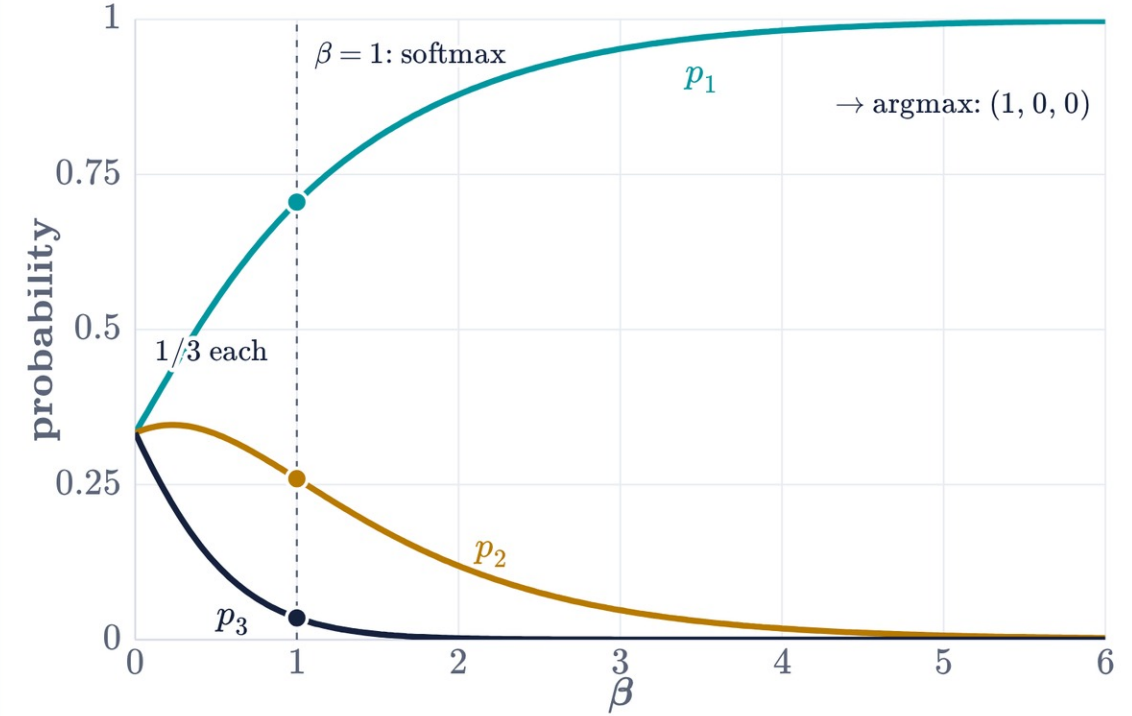
BONUS

Softmax vs argmax

- The **argmax** puts all the probability on the top score: $(1, 0, 0)$ for $\boldsymbol{z} = (2, 1, -1)$. It jumps when two scores swap.
- Dividing by the sum also gives numbers that add to 1, but only for positive scores, and it keeps the ratios: $(3, 2, 1)$ gives $(0.50, 0.33, 0.17)$. The softmax gives $(0.67, 0.24, 0.09)$: the exponential stretches the gaps.
- Put a coefficient β in front of the scores:

$$p_k = \frac{e^{\beta z_k}}{\sum_j e^{\beta z_j}}$$

- $\beta = 0$: every class equal. $\beta = 1$: the softmax. $\beta \to \infty$: the argmax.



The scores $\boldsymbol{z} = (2, 1, -1)$ from the softmax slide.

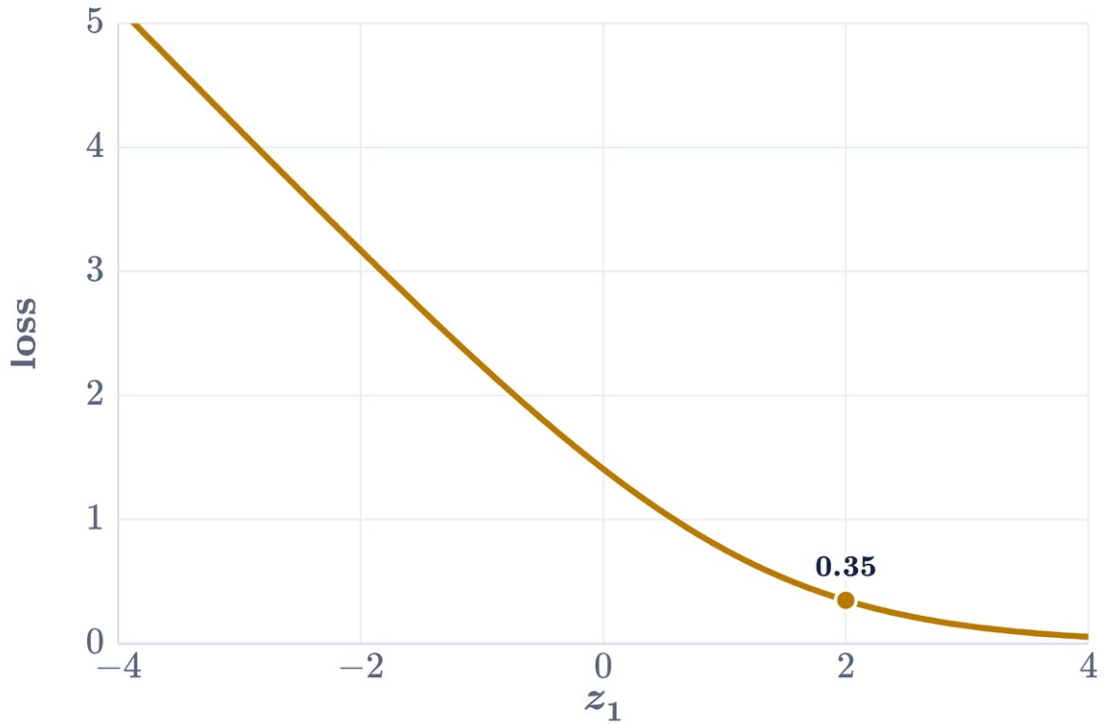
BONUS

Softmax and cross-entropy fit together

- For a point of class c, the log undoes the softmax's exponential:

$$\mathcal{L} = -\log \frac{e^{z_c}}{\sum_j e^{z_j}}$$

$$= -z_c + \log \sum_j e^{z_j}$$



Class 1 is the true class; the other scores are fixed at $z_2 = 1$, $z_3 = -1$.

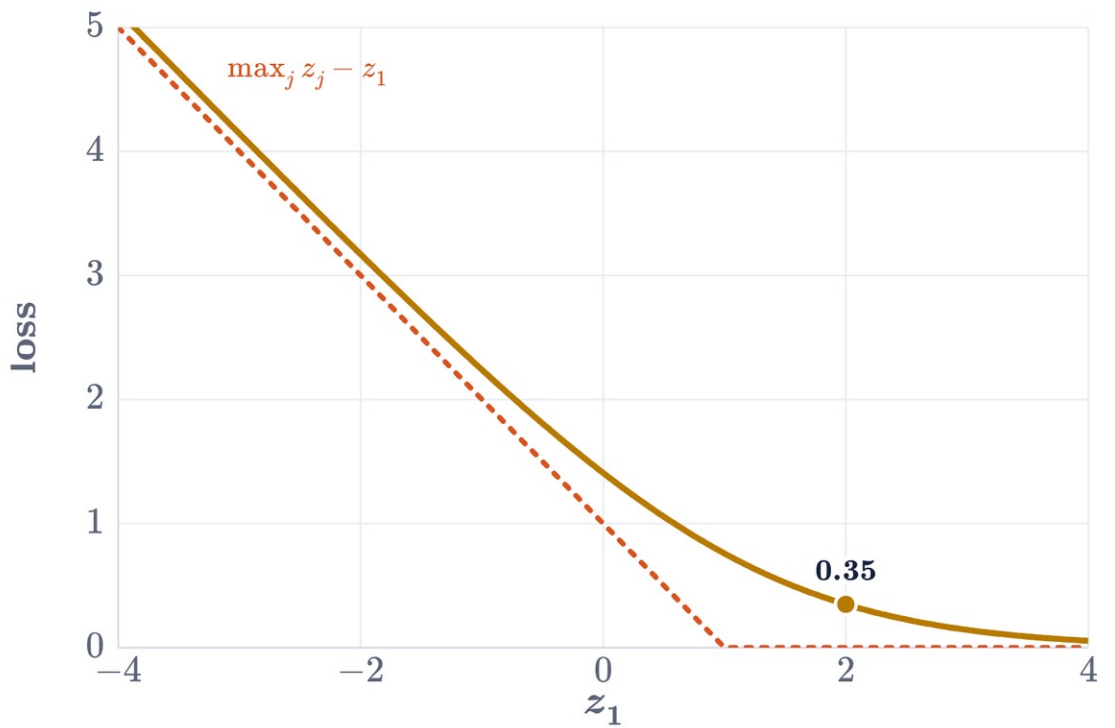
BONUS

Softmax and cross-entropy fit together

- For a point of class c, the log undoes the softmax's exponential:

$$\mathcal{L} = -\log \frac{e^{z_c}}{\sum_j e^{z_j}}$$
$$= -z_c + \log \sum_j e^{z_j}$$

- $\log \sum_j e^{z_j}$ is a smooth version of $\max_j z_j$. So the loss is about how far the true class's score is behind the top score.



Class 1 is the true class; the other scores are fixed at $z_2 = 1$, $z_3 = -1$.

BONUS

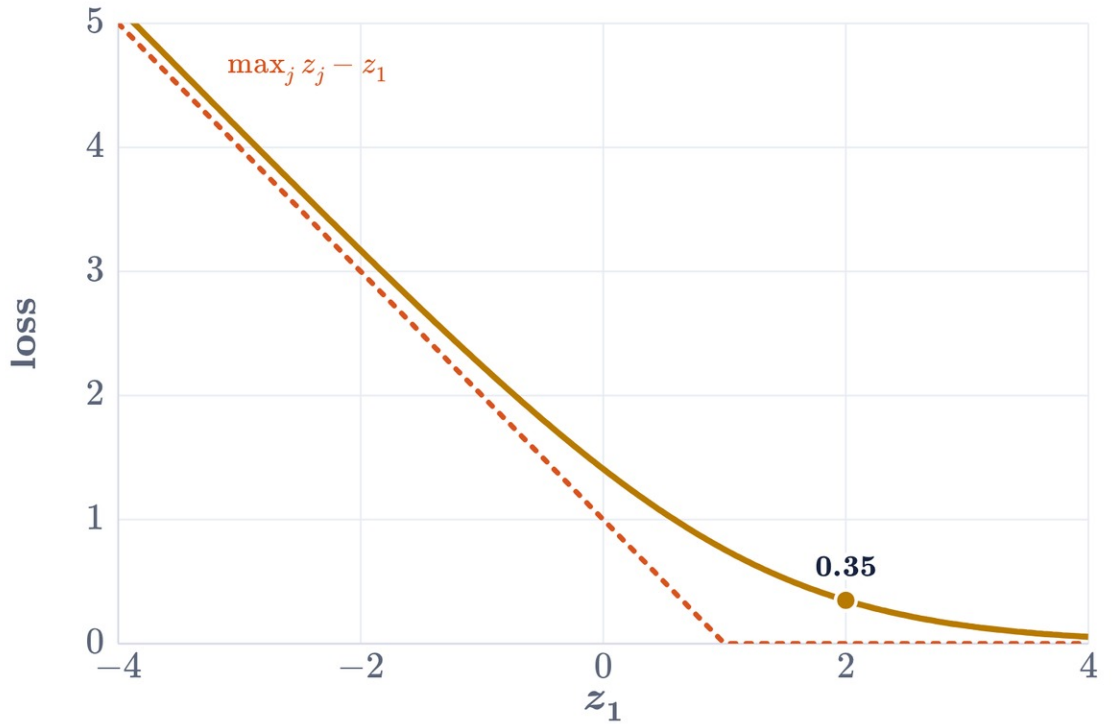
Softmax and cross-entropy fit together

- For a point of class c, the log undoes the softmax's exponential:

$$\mathcal{L} = -\log \frac{e^{z_c}}{\sum_j e^{z_j}}$$
$$= -z_c + \log \sum_j e^{z_j}$$

- $\log \sum_j e^{z_j}$ is a smooth version of $\max_j z_j$. So the loss is about how far the true class's score is behind the top score.
- The derivative is prediction minus label:

$$\frac{\partial \mathcal{L}}{\partial z_k} = p_k - y_k$$



Class 1 is the true class; the other scores are fixed at $z_2 = 1$, $z_3 = -1$.

BONUS

Softmax and cross-entropy fit together

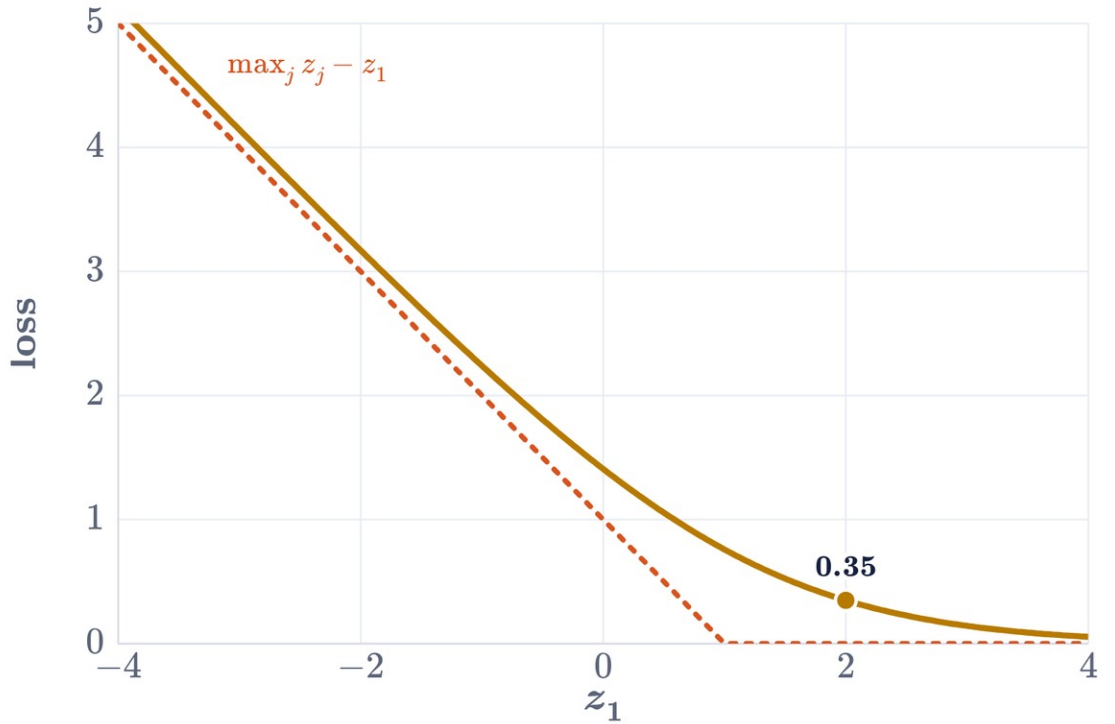
- For a point of class c, the log undoes the softmax's exponential:

$$\mathcal{L} = -\log \frac{e^{z_c}}{\sum_j e^{z_j}}$$
$$= -z_c + \log \sum_j e^{z_j}$$

- $\log \sum_j e^{z_j}$ is a smooth version of $\max_j z_j$. So the loss is about how far the true class's score is behind the top score.
- The derivative is prediction minus label:

$$\frac{\partial \mathcal{L}}{\partial z_k} = p_k - y_k$$

- With two classes and scores $(f, 0)$, class 1 gives $\mathcal{L} = \log(1 + e^{-f})$: the log loss.



Class 1 is the true class; the other scores are fixed at $z_2 = 1$, $z_3 = -1$.